

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گروه مهندسی نرم افزار

گزارش پروژه کارشناسی

رشته مهندسی کامپیوتر گرایش نرم افزار

عنوان پروژه:

تشخیص ناهنجاری در لاگ های شبکه از طریق مصورسازی آن ها

استاد راهنما:

دکتر حمیدرضا برادران کاشانی

پژوهشگر:

نگار باروتی

شهریور ۱۴۰۲



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گروه مهندسی نرم افزار

پروژه کارشناسی رشته‌ی مهندسی کامپیوتر گرایش نرم افزار

خانم نگار باروتی

تحت عنوان

تشخیص ناهنجاری در لاگ‌های شبکه از طریق مصورسازی آن‌ها

در تاریخ / / ۱۴۰۲ توسط هیأت داوران زیر بررسی و با نمره به تصویب نهایی رسید.

۱- استاد راهنمای پروژه:

دکتر حمیدرضا برادران کاشانی

امضا

۲- استاد داور:

دکتر

امضا

امضای مدیر گروه

چکیده:

لاگ شبکه^۱ حاوی اطلاعات ارزشمندی برای تشخیص رفتار غیرمنتظره یا غیرعادی در یک شبکه با مقیاس بزرگ است. لاگ‌ها توسط مجموعه بزرگی از دستگاه‌های شبکه (مانند روترها، سوئیچ‌ها و سرورها) تولید می‌شوند و از حجم عظیمی برخوردارند، به همین علت مشخص کردن خرابی‌ها و دلایل آن، چالش آسانی برای اپراتورهای شبکه نیست. مصورسازی^۲ داده‌های دریافتی از این گزارش‌های متنی و شناسایی ناهنجاری‌های موجود در میان آنها تا حدی این مشکل را برطرف خواهد کرد. مصورسازی داده‌های متنی به معنای نمایش و اشتراک‌گذاری اطلاعات متنی به صورت قابل فهم، از طریق نمودارها و نقشه‌ها می‌باشد. مصورسازی لاگ‌های شبکه، به منظور ارائه یک نمایش ساده و مختصر از ترافیک و فعالیت‌های شبکه است. هر بار که رویدادی خاص در سیستم شما رخ می‌دهد به صورت اتوماتیک لاگ آن رویداد به همراه مهر زمانی^۳ آن، تولید می‌شود. در نتیجه حجم لاگ‌های سیستم بسیار بالا است. این داده‌ها به علت وقوع پشت‌سرم در دسته‌ی داده‌های متوالی^۴ قرار می‌گیرند. بررسی و مطالعه فایل‌های لاگ باید توسط فرد متخصص صورت گیرد و یا از طریق مصورسازی و به کمک نمودارها و گراف‌ها، آنها را برای کاربر عادی قابل فهم کرد. از دیگر کاربردهای مصورسازی می‌توان به ساخت روندها و الگوهای تکراری^۵ برای درک بهتر علت وقوع لاگ و نحوه برخورد با آن اشاره کرد. بعد از بیان کاربردها و مزایای مصورسازی باید در نظر داشت که ارائه تصویری داده‌ها به تنهایی برای کاربر عادی، ممکن است سودبخش نباشد و یا نیاز است که کاربر عادی لاگ‌های جدیدی که دریافت می‌کند را با لاگ‌های گذشته سیستم خود مقایسه کند تا خرابی‌ها و یا عوامل مخرب سیستم خود را پیدا کند، از بروز حملات مخرب به سیستم خود جلوگیری کند، امنیت سیستم را بالا ببرد و یا حتی در مواردی عملکرد سیستم را بهبود بخشد، در چنین مواردی بهتر است به سراغ الگوریتم‌های تشخیص ناهنجاری برویم.

واژگان کلیدی: لاگ، شبکه، مصورسازی، داده، متن

^۱ Network log

^۲ Visualization

^۳ Timestamp

^۴ Sequence Data

^۵ Frequent Pattern

فهرست مطالب

عنوان	صفحه
فصل اول مقدمه	۵
۱-۱- هدف پروژه	۵
۲-۱- کاربردهای پروژه	۵
۳-۱- ساختار پایان نامه	۶
فصل دوم مفاهیم	۸
۱-۲- مقدمه	۸
۲-۲- ماتریس هیستوگرام	۸
۳-۲- ماتریس شباهت	۱۱
۴-۲- مدل ماشین بردار پشتیبان	۱۵
۵-۲- شبکه RNN	۱۶
۶-۲- شبکه LSTM	۱۹
۷-۲- الگوریتم کاهش بعد T-SNE	۲۱
۸-۲- جمع بندی	۲۲
فصل سوم شرح پروژه	۲۳
۱-۳- مقدمه	۲۳
۲-۳- پیاده سازی ماتریس شباهت	۲۳
۳-۳- پیاده سازی مدل های یادگیری عمیق	۳۰
۴-۳- مدل mlp	۳۳
۵-۳- مدل RNN	۳۵
۶-۳- مدل LSTM	۳۸
۷-۳- ارزیابی مدل های یادگیری عمیق	۴۰

فهرست مطالب

عنوان	صفحه
۳-۸- پیاده سازی بهترین مدل برای مصورسازی بر روی دیتاست لاگ	۴۱
۳-۹- جمع بندی	۴۸
فصل چهارم نتایج	۴۹
۴-۱- مقدمه	۴۹
۴-۲- نتایج مصورسازی با ماتریس شباهت	۴۹
۴-۳- نتایج مصورسازی حالت های پنهان	۵۰
۴-۴- جمع بندی	۵۴
فصل پنجم نتیجه گیری و پیشنهادها	۵۵
۵-۱- پیشنهادات در صورت استفاده از ماتریس شباهت	۵۵
۵-۲- پیشنهادات در صورت استفاده از مصورسازی حالت های پنهان	۵۵
منابع:	۵۷

فهرست شکل‌ها

عنوان	صفحه
شکل ۱ ماتریس هیستوگرام	۱۰
شکل ۲ الگوریتم SLCT برای دسته‌بندی لاگ‌ها	۱۱
شکل ۳ نمونه ای از ماتریس شباهت (عنوانین عمودی در سمت چپ تصویر بیانگر ویژگی‌های داده هاست و محور افقی عنوان هر داده نمونه است)	۱۴
شکل ۴ مرز تصمیم‌گیر محاسبه شده توسط مدل به همراه حاشیه و نقاط اطمینان	۱۶
شکل ۵ گراف دوبخشی میان حالت‌های پنهان و ورودی‌های مجزا در جمله	۱۸
شکل ۶ دسته‌بندی حالت‌های پنهان بر اساس ورودی	۱۹
شکل ۷ فضای دوبعدی برای نمایش تمام حالت‌های پنهان لایه lstm مدل (این کاهش بعد به کمک tsne انجام گرفته است)	۲۲
شکل ۸ داده‌های اسمی	۲۴
شکل ۹ چند نمونه از ویژگی‌هایی که از مقادیر اسمی به عددی تبدیل شده‌اند	۲۴
شکل ۱۰ برچسب‌گذاری عددی داده‌ها به صورت ۰ و ۱	۲۵
شکل ۱۱ پیش‌بینی داده‌های تست توسط مدل svm	۲۶
شکل ۱۲ پیش‌بینی مدل svm برای داده‌های تست	۲۶
شکل ۱۳ ارزیابی مدل svm	۲۷
شکل ۱۴ محاسبه فاصله ۱۲ ویژگی مهم از ۱۰ نمونه از دیتاست تست با مرز تصمیم‌گیر مدل svm	۲۸
شکل ۱۵ ماتریس شباهت (فاصله مقادیر ویژگی‌ها برای هر داده نمونه با مرز تصمیم‌گیر در همان بعد مربوط به آن ویژگی)	۲۹
شکل ۱۶ ماتریس شباهت برای ۶ ویژگی بعدی	۳۰
شکل ۱۷ زیرمدل مربوط به پیش‌بینی مدل	۳۱
شکل ۱۸ کاهش بعد توسط tsne برای مصورسازی حالت‌های پنهان	۳۲
شکل ۱۹ ساختار کلی مدل mlp	۳۳
شکل ۲۰ دقت مدل mlp	۳۳
شکل ۲۱ نتیجه مصورسازی ابرپارامترهای مدل mlp	۳۴
شکل ۲۲ مصورسازی یک داده نمونه با mlp و نمایش نامناسب رنگ برای کلمات ورودی	۳۵
شکل ۲۳ آموزش با مدل rnn	۳۶
شکل ۲۴ مصورسازی یک داده نمونه با برچسب منفی با مدل rnn	۳۷

فهرست شکل‌ها

عنوان	صفحه
شکل ۲۵ مصورسازی با مدل rnn برای یک داده نمونه با برچسب مثبت	۳۸
شکل ۲۶ ساختار مدل lstm	۳۹
شکل ۲۷ آموزش و پیش‌بینی مدل lstm	۳۹
شکل ۲۸ مصورسازی نهایی مدل lstm	۴۰
شکل ۲۹ مصورسازی دیتاست آموزش به کمک tsn-e	۴۱
شکل ۳۰ توضیح برخی از ستون‌های دیتاست zookeeper	۴۲
شکل ۳۱ نمایش اطلاعات لاگ با جزئیات مقادیر	۴۳
شکل ۳۲ ساختار مدل lstm به تفکیک دو زیر مدل مجزای آن شامل مصورسازی و پیش‌بینی	۴۴
شکل ۳۳ آموزش مدل lstm بر روی دیتاست لاگ	۴۵
شکل ۳۴ مصورسازی ۱۰۰ داده نمونه تست از دیتاست zookeeper	۴۶
شکل ۳۵ مصورسازی برای داده نمونه با برچسب هشدار	۴۷
شکل ۳۶ مصورسازی برای داده نمونه با برچسب اطلاعات	۴۷
شکل ۳۷ مصورسازی کلمات درون لاگ به همراه شدت رنگ آنها	۴۸
شکل ۳۸ بررسی نتایج ماتریس شباهت	۵۰
شکل ۳۹ مصورسازی حالت‌های پنهان لایه Bidirectional	۵۱
شکل ۴۰ مصورسازی حالت‌های پنهان در لایه gru	۵۲
شکل ۴۱ مصورسازی حالت‌های پنهان لایه پیچشی	۵۲
شکل ۴۲ ساختار مدل شبکه عصبی پیچشی به همراه آموزش و دقت آن	۵۳
شکل ۴۳ مصورسازی حالت‌های پنهان با الگوریتم کاهش بعد pca	۵۴

فصل اول

مقدمه

۱-۱- هدف پروژه

در این پروژه ابتدا قصد داریم با بررسی ابزارهای مصورسازی موجود، ابزار مناسب جهت ارائه ساده و قابل فهم داده‌های لاگ را توسعه دهیم. علاوه بر آن به تشخیص ناهنجاریها در میان لاگهای دریافتی خواهیم پرداخت. فرایند مصورسازی داده میتواند بعد از تشخیص ناهنجاری انجام شود و برای درک بهتر، خروجی مدل مصور شود. حالت دیگر این است که مصورسازی داده قبل از تشخیص ناهنجاری انجام شود. در این حالت با مصورسازی میتوان ناهنجاری را تعیین کرد.

۱-۲- کاربردهای پروژه

۱-۲-۱- امنیت شبکه

این پروژه به مدیران شبکه و تیمهای امنیت کمک می‌کند تا به صورت خودکار و بهینه تر اقدامات امنیتی را برای تشخیص و جلوگیری از حملات ناهنجار انجام دهند. از جمله کاربردها می‌توان به تشخیص حملات DDOS، تلاش‌های نفوذ، کاربران مشکوک و... اشاره کرد.

۱-۲-۲- پیش‌بینی خرابی‌ها

با تحلیل داده‌های لاگ شبکه، می‌توان به پیش‌بینی خرابی‌ها و اختلالات در شبکه پرداخت. این امر به مدیران امکان می‌دهد تا قبل از وقوع مشکلات، تدابیر لازم را اتخاذ کنند.

۱-۲-۳- مانیتورینگ عملکرد شبکه

مصورسازی داده‌های لاگ شبکه، به مدیران کمک می‌کند تا عملکرد شبکه را به صورت دقیق‌تر و در زمان واقعی مشاهده کنند. این کاربرد به شناسایی مشکلات احتمالی در عملکرد شبکه کمک می‌کند.

۱-۲-۴- تجزیه و تحلیل رفتار کاربران

با تحلیل لاگ‌های شبکه، می‌توان الگوهای رفتاری کاربران را تشخیص داد و در صورت وجود تغییرات ناهنجار، به تشخیص دست‌کمی‌ها و نقض‌های امنیتی پرداخت.

۱-۲-۵- بهبود عملکرد و بهینه‌سازی شبکه

از طریق تحلیل لاگ‌های شبکه و مصورسازی داده‌ها، می‌توان مشکلات عملکردی و منابع مصرفی زیاد را شناسایی کرده و اقدامات بهبودی را اجرا کرد.

۱-۲-۶- پژوهش‌های تحلیل داده

این پروژه می‌تواند به پژوهش‌های مرتبط با تحلیل داده‌های شبکه، تشخیص ناهنجاری و یادگیری ماشین مرتبط با امنیت شبکه کمک کند و محققان را در جهت بهبود روش‌ها و الگوریتم‌ها هدایت کند.

۱-۳- ساختار پایان‌نامه

در فصل دوم ابتدا به سراغ بررسی مقاله‌هایی که بر روی داده‌های لاگ و شبکه کار کرده‌اند می‌رویم و مفاهیم مرتبط در این زمینه را به صورت کلی توضیح خواهیم داد. در فصل سوم از میان مقاله‌های بررسی شده، ایده ماتریس شباهت را که برگرفته از یکی از مقالات می‌باشد را مدنظر قرار داده و به کمک یکی از مدل‌های یادگیری ماشین، مدلی را طراحی کرده و پس از آموزش مدل به سراغ مصورسازی داده‌های تست می‌رویم و تلاش می‌کنیم با بررسی الگوهای تکراری، نمونه داده‌ی ناهنجار در میان داده‌های تست را پیدا کرده و حتی ویژگی مؤثر بر وقوع این ناهنجاری را نیز تا حدودی مشخص کنیم.

علاوه بر این در فصل سوم سعی خواهیم کرد با بررسی مقالات جدیدتر به سراغ مدل‌های مبتنی بر یادگیری عمیق برویم. در فصل سوم دو گونه مصورسازی را مورد بررسی قرار خواهیم داد در حالت اول ابتدا مدل را آموزش داده و مصورسازی را بر روی داده تست خواهیم داشت که ببینیم مدل تا چه حد نمونه داده ناهنجار را تشخیص داده است؛ اما در حالت دوم پس از آموزش مدل، ابر پارامترهای آموزش داده شده توسط مدل را مصورسازی خواهیم کرد.

در پایان فصل سوم از میان مدل‌های مبتنی بر یادگیری عمیق و مصورسازی‌های متفاوت آنها جدیدترین مدل مصورسازی را انتخاب خواهیم کرد و با ثابت نگه داشتن نحوه مصورسازی سعی خواهیم کرد مدل را تغییر داده تا بتوان نتایج بهتر کسب کرد و مدل را از نحوه مصورسازی مستقل کرد که بعدها نیز بتوان مدل را بسط داده یا جدیدتر کنیم.

در فصل چهارم نتایج به دست آمده در حالت‌های مختلف را با انواع مصورسازی‌های صورت گرفته بررسی خواهیم کرد.

در فصل پنجم نیز به نتیجه گیری و راه‌های پیشرفت و ادامه‌ی پروژه می‌پردازیم.

فصل دوم

مفاهیم

۲-۱- مقدمه

در این فصل ابتدا مفاهیم موجود در چند مقاله مرتبط با مصورسازی لاگ‌ها را بررسی می‌کنیم. در طی این فصل ابتدا الگوریتم‌های یادگیری ماشین را پیش می‌بریم و مصورسازی داده‌های تست را در راستای تشخیص ناهنجاری انجام می‌دهیم. بعد از آن به سراغ الگوریتم‌های یادگیری عمیق خواهیم رفت و مصورسازی مدل و ابرپارامترهای آن را خواهیم داشت.

۲-۲- ماتریس هیستوگرام

مقاله [۱] به موضوع تجزیه و تحلیل فایل‌های لاگ پرداخته و یک تکنیک نوآورانه برای تجزیه و تحلیل این فایل‌ها به نام $HMAT^1$ را ارائه می‌دهد. هدف اصلی این تکنیک، تجزیه و تحلیل محتوای فایل‌های لاگ به منظور شناسایی نقض‌ها و ناهنجاری‌ها به صورت مؤثر است. در مقاله به توضیح این تکنیک و نحوه کارکرد آن پرداخته شده است.

تکنیک $HMAT$ با استفاده از ترکیبی از روش‌های گرافیکی و آماری، محتوای یک فایل لاگ را به تصویر

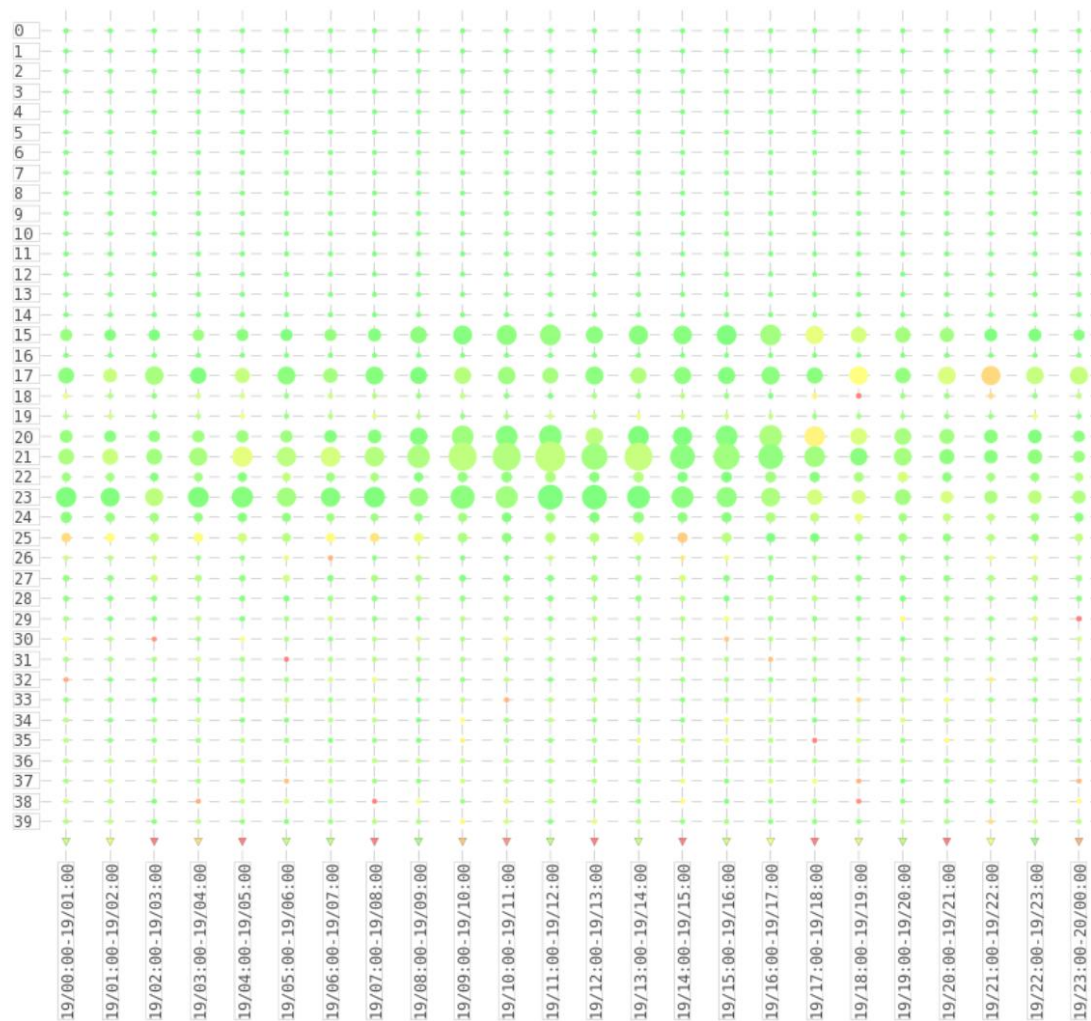
¹ Histogram Matrix

می‌کشد تا به مدیران امنیت کمک کند تا به‌صورت کارآمد نقض‌ها را تشخیص دهند. این تکنیک به کمک ابزارهای گرافیکی و آماری، حتی به افراد غیرمتخصص اجازه می‌دهد تا به‌صورت تعاملی به جستجوی پیام‌های لاگ ناهنجار بپردازند. یکی از ویژگی‌های مهم این تکنیک این است که به شکلی عمومی بر روی انواع مختلفی از فایل‌های لاگ عمل می‌کند، نه فقط نوع‌های خاصی از آن‌ها.

در این مقاله، HMAT به‌صورت ماتریسی تأخیر پیشرفت ویژگی‌های مشخصی از یک فایل لاگ را در طول زمان نمایش می‌دهد. این ویژگی‌ها شامل تعداد کلمات و تعداد کاراکترها در هر پیام لاگ هستند. این ماتریس توزیع این ویژگی‌ها را در یک بازه زمانی خاص (مثلاً یک روز) نمایش می‌دهد. همچنین، اطلاعات آماری از پیام‌های لاگ گذشته نیز نگهداری می‌شوند و برای تشخیص ازدست‌رفتن توزیع عادی ویژگی‌ها استفاده می‌شوند که این مهم می‌تواند تا حدودی ناهنجاری را نیز شناسایی کند.

نحوه خوشه‌بندی لاگ‌ها در این ماتریس به کمک الگوریتم SLCT است که بر اساس پرتکرار بودن الگوهای ساختاری در میان لاگ‌ها می‌باشد. به این صورت که کلمات نادر و کم تکرار با یک ستاره * جایگزین می‌کند. (شکل ۲)

در واقع برای ترسیم ماتریس هیستوگرام ابتدا لاگ‌ها را بر اساس تعداد کلمه موجود در آن دسته‌بندی می‌کنیم. سپس ماتریسی رسم می‌کنیم که عنوان هر سطر آن نشان‌دهنده شماره دسته‌ی مربوط به لاگ‌ها است و عنوان هر ستون تاریخ وقوع لاگ‌های موجود در آن دسته است. (شکل ۱)



شکل ۱ ماتریس هیستوگرام

540	2007-04-22 * * => * R=zhaw.mailhub T=remote.smtp H=orwell.zhaw.ch [160.85.104.31]
120	2007-04-22 * 1HfNjC-000621-MW => * R=dnslookup T=remote.smtp * *
91	2007-04-22 * 1HfNjC-0000hH-7x => * R=dnslookup T=remote.smtp * *
81	2007-04-22 * * => * R=dnslookup T=remote.smtp * *
75	2007-04-22 * 1HfNjC-000621-MW -> * R=dnslookup T=remote.smtp * *
67	2007-04-22 * 1HfNjC-0000hH-7x -> * R=dnslookup T=remote.smtp * *
31	2007-04-22 00:11:44 1HfNjC-0000hH-7x -> * R=dnslookup T=remote.smtp H=mxbw.bluewin.ch [195.186.18.144]
30	2007-04-22 * * => user@zhaw.ch R=zhaw.mailhub T=remote.smtp H=orwell.zhaw.ch [160.85.104.31]
...	...

Table 2. Clusters of anomaly 1.

miro

شکل ۲ الگوریتم SLCT برای دسته‌بندی لاگ‌ها

دیتاستی که این مقاله مورد بررسی قرار داده است مربوط به سیستم میل سرور دانشگاهی می‌باشد و شامل لاگ‌های جمع‌آوری شده از این شبکه در یک روز بدون حادثه است.

مقاله فوق به توضیح مفاهیم و روش‌های مرتبط با تجزیه و تحلیل فایل‌های لاگ می‌پردازد و تکنیک HMAT را به عنوان یک راهکار نوآورانه برای تجزیه و تحلیل این فایل‌ها معرفی می‌کند، اما از آن جهت که خوشه‌بندی داده‌ها بر روی تعداد کلمه موجود در لاگ‌ها اتفاق افتاده است، در صورت وجود ساختار یکسان در میان لاگ‌ها این الگوریتم به خوبی عمل نخواهد کرد. به همین منظور ما به دنبال روش دیگری برای مصورسازی و تشخیص ناهنجاری رفته‌ایم.

۲-۳- ماتریس شباهت

ایده ماتریس شباهت از مقاله [۲] گرفته شده است. پس از درک عملکرد ماتریس شباهت برای بررسی عملکرد آن در فصل سوم به سراغ پیاده‌سازی آن با آموزش مدل svm روی دیتاست مشهور NSL-KDD که مربوط به لاگ‌های شبکه است، می‌رویم.

این دیتاست بر اساس لاگ‌ها را بر اساس نوع وقوع حادثه‌ای که در شبکه رخ داده است و می‌تواند به صورت طبیعی یا غیرطبیعی باشد، به دو دسته ۰ و ۱ برچسب‌گذاری می‌کند، ۰ به معنای طبیعی بودن آن رخداد در شبکه است و ۱ به معنای غیرطبیعی بودن آن رخداد است.

از دیتاست NSL-KDD برای ترسیم ماتریس شباهت استفاده شده و از آن جهت که این مجموعه داده در حدود ۴۰ ویژگی را پوشش می‌دهد ما چند ویژگی مهم‌تر آن را توضیح خواهیم داد و در مدل نیز از این ویژگی‌ها استفاده خواهیم کرد.

از ویژگی‌های مطرح شده در این دیتاست می‌توان به موارد زیر اشاره کرد:

- مدت‌زمان^۱ برای انجام اتصال یا فعالیت شبکه
- نوع پروتکل^۲ که می‌تواند tcp, udp, icmp باشد.
- نوع سرویس^۳ یا برنامه‌ای که در شبکه استفاده می‌شود.
- پرچم^۴ که نشانگر وضعیت اتصال می‌باشد و sf به معنای اتصال خوب و so به معنای اتصال بسته شده و rej به معنای اتصال رد شده می‌باشد.
- تعداد بایت‌های ارسالی از فرستنده^۵
- تعداد بایت‌های دریافتی توسط گیرنده^۶
- داغ^۷: این ویژگی نیز به نام نشانگر داغ^۸ نیز شناخته می‌شود. این ویژگی نشان می‌دهد که آیا مقصد موردنظر به‌عنوان یک میزبان داغ شناخته می‌شود یا خیر. در صورتی که یک میزبان در یک مدت‌زمان کوتاه تعداد بسیار زیادی از اتصالات را بپذیرد، به‌عنوان میزبان داغ شناخته می‌شود. این ویژگی اطلاعاتی درباره الگوهای ترافیکی مخاطب مقصد فراهم می‌کند.
- تعداد کل اتصالات^۹ که با میزبان مقصد موردنظر برقرار شده است. این ویژگی به تعداد کل ارتباط‌هایی اشاره دارد که با میزبان مقصد در طی مدت‌زمان مشخص شده برقرار شده است.

^۱ duration

^۲ protocol type

^۳ service

^۴ flag

^۵ src-bytes

^۶ dst-bytes

^۷ hot

^۸ Hot indicato

^۹ count

- تعداد اتصالات بازاریابی^۱ (برای انجام خدمات مشخصی) که با میزبان مقصد موردنظر برقرار شده است.
 - نسبت اتصالات بازاریابی (برای انجام خدمات مشخصی) به کل اتصالات^۲ با میزبان مقصد مشخص شده در ویژگی‌های قبلی. این ویژگی نشان می‌دهد که چه مقدار از اتصالات با میزبان مقصد موردنظر به‌منظور ارتباط بازاریابی (خدمات مشخصی) انجام شده است.
 - تعداد اتصالات بازاریابی^۳ (برای انجام خدمات مشخصی) که با میزبان مقصد موردنظر برقرار شده است.
 - نسبت اتصالات بازاریابی (برای انجام خدمات مشخصی) به کل اتصالات^۴ با میزبان مقصد مشخص شده در ویژگی‌های قبلی برای میزبان مقصد موردنظر.
 - نسبت اتصالات مختلف (غیر بازاریابی) به کل اتصالات^۵ با میزبان مقصد مشخص شده در ویژگی‌های قبلی برای میزبان مقصد موردنظر.
 - نسبت اتصالات با میزبان مقصد^۶ موردنظر که از یک پورت مبدأ خاص به‌عنوان منبع برقرار شده‌اند به کل اتصالات با میزبان مقصد موردنظر.
- برای ساخت این ماتریس ابتدا فاصله هر داده نمونه در دیتاست تست را با مرز تصمیم‌گیری به وجود آمده در مدل محاسبه می‌کنیم. به‌نحوی که عدد قرار گرفته در هر سطر و ستون بیانگر میزان فاصله نمونه داده با مرز تصمیم‌گیری در یک بعد مشخص است تا بتوان برای هر ویژگی این فاصله را محاسبه کرد. برای مثال سطر اول این ماتریس میزان فاصله داده با مرز تصمیم‌گیر در بعد اول است و نشان‌دهنده تأثیرگذاری ویژگی نخست دیتاست بر روی برچسب پیش‌بینی‌شده توسط مدل برای هر نمونه داده است. (شکل ۴)

^۱ srv-count

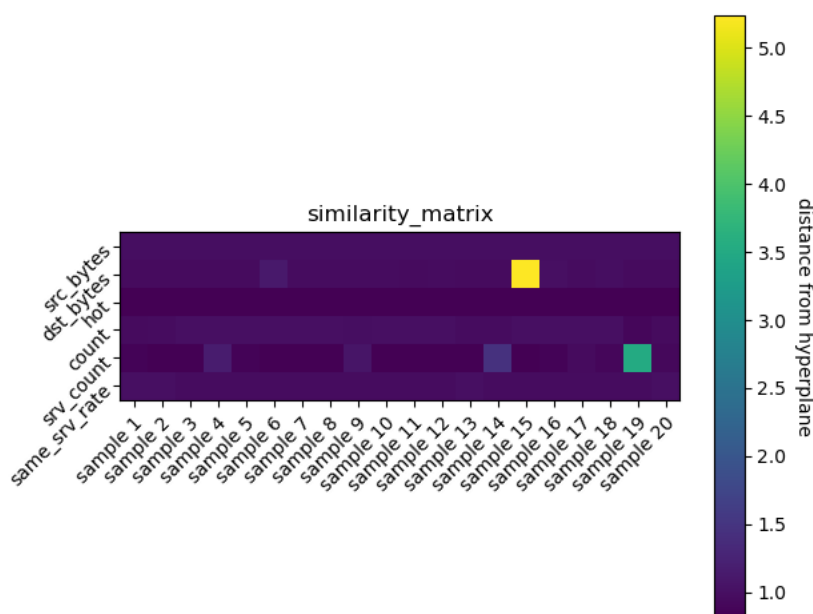
^۲ same-srv-rate

^۳ dst-host-srv-count

^۴ dst-host-same-srv-rate

^۵ dst-host-diff-srv-rate

^۶ dst-host-same-src-port-rate



شکل ۳ نمونه ای از ماتریس شباهت (عنوانین عمودی در سمت چپ تصویر بیانگر ویژگی‌های داده هاست و محور افقی عنوان هر داده نمونه است)

دیتاست مورد استفاده در خود مقاله شامل برچسب‌های بدافزار^۱ و بی خطر^۲ می‌باشد. این مقاله پس از آموزش مدل و مشخص شدن مرز تصمیم‌گیر به سراغ رسم ماتریس شباهت خواهد رفت. این ماتریس را یکبار بر روی داده‌هایی که برچسب بی خطر دارند محاسبه خواهد کرد و بار دیگر بر روی داده‌های بدافزار و بار سوم ماتریس را برای داده‌های بدافزار و بی خطر باهم ترسیم می‌کند و مشاهده می‌کند که شباهت داده‌های بدافزار با یکدیگر زیاده‌تر از حالت‌های دیگر است در نتیجه اگر داده‌ی تست جدیدی بیاید می‌توان ماتریس شباهت یکبار با داده‌های بدافزار و بار دیگر با داده‌های بی خطر ترسیم کرد و در صورت یافتن شباهت با داده بدافزار داده تست جدید را بدافزار در نظر گرفت.

^۱ malware

^۲ benign

۲-۴- مدل ماشین بردار پشتیبان^۱

یک الگوریتم یادگیری ماشین است که برای مسائل دسته‌بندی و رگرسیون مورد استفاده قرار می‌گیرد. به‌عنوان یک مدل ماشینی از داده‌های آموزشی یاد می‌گیرد و سپس با استفاده از دانش حاصل، قادر به پیش‌بینی و تصمیم‌گیری در مورد داده‌های جدید است.

در دسته‌بندی، SVM به دنبال یافتن یک مرز تصمیم‌گیر^۲ است که بتواند بین داده‌های دو دسته مختلف مانند یک تخته جداکننده، قرار گیرد. مرز تصمیم‌گیر به نحوی انتخاب می‌شود که فاصله کمینه از نمونه‌های دو دسته به آن بیشینه شود. این فاصله که به نام حاشیه^۳ شناخته می‌شود، تا حد امکان انحراف‌ها و اشتباهات در تصمیم‌گیری را کاهش می‌دهد. (شکل ۳)

در مدل SVM دو ویژگی مهم به نام‌های `_coef` و `_intercept` وجود دارند که در فرایند تعیین مرز تصمیم‌گیری توسط SVM نقش دارند.

• `_coef` :

این ویژگی، ضرایب مرتبط با داده‌های ورودی را نشان می‌دهد. در مدل SVM خطی، هر ویژگی ورودی با یک ضریب مرتبط است. این ضرایب نشان‌دهنده وزن‌دهی به هر ویژگی در فرایند تصمیم‌گیری دسته‌بندی هستند. در ادامه و با محاسبه فاصله وزن هر ویژگی با مقدار آن ویژگی در هر داده نمونه می‌توان تأثیرگذاری آن ویژگی در برچسب پیش‌بینی‌شده توسط مدل را دریافت.

• `_intercept` :

این ویژگی حاشیه تصمیم‌گیری را نشان می‌دهد. در مدل SVM خطی، این مقدار نشان‌دهنده فاصله مرتبط با مرز تصمیم‌گیری از مبدأ است. در مدل دسته‌بندی دو کلاسه، مقدار `_intercept` تنها برای کلاس مثبت یا کلاس یکی از دو کلاس وجود دارد.

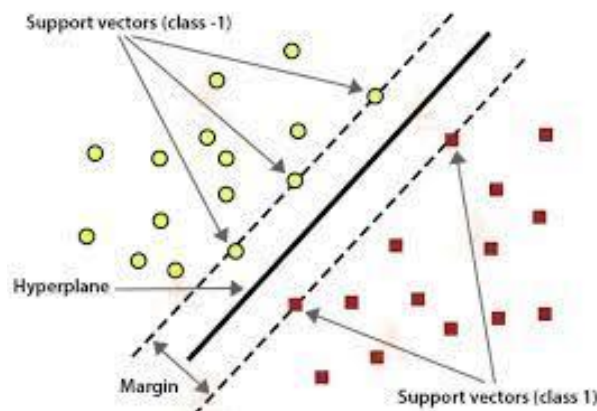
این دو ویژگی `_coef` و `_intercept` از اهمیت بالایی برخوردار هستند چرا که نشان می‌دهند که چگونه مدل SVM تصمیم‌گیری را انجام می‌دهد و چگونه داده‌ها به دسته‌های مختلف تخصیص داده می‌شوند. بر اساس این دو ویژگی، می‌توان تحلیل‌هایی در مورد تأثیر هر ویژگی بر تصمیم‌گیری مدل و همچنین فاصله میان

^۱ SVM

^۲ hyperplane

^۳ margin

مرزهای تصمیم‌گیری انجام داد. در پیاده‌سازی ماتریس شباهت نیز از این‌گونه تحلیل‌ها برای ترسیم ماتریس استفاده خواهیم کرد و ویژگی‌های مؤثر بر وقوع داده‌های ناهنجار را به این روش مشخص می‌کنیم.



شکل ۴ مرز تصمیم‌گیر محاسبه شده توسط مدل به همراه حاشیه و نقاط اطمینان

در حالتی که داده‌ها قابل تفکیک خطی نیستند، SVM از تبدیلات ریاضی به فضای بالاتر استفاده می‌کند تا داده‌ها را به یک فضای دوبعدی تبدیل کند که در آن تفکیک خطی ممکن شود. این تبدیلات به نام تابع هسته^۱ انجام می‌شوند.

مزیت‌های اصلی مدل SVM عبارت‌اند از:

- قابلیت کاربرد در دسته‌بندی داده‌های خطی و غیرخطی با استفاده از تابع‌های هسته.
- مقاومت در برابر ابعاد بالا^۲.

اما محدودیت‌هایی مانند حساسیت به پارامترهای تنظیمی، مصرف منابع محاسباتی بالا و پیچیدگی در انتخاب تابع هسته دارد. در کل، SVM یک ابزار قدرتمند در دسته‌بندی و پیش‌بینی با عملکرد خوب و قابلیت کاربرد گسترده است.

۲-۵- شبکه RNN^۳

در موارد قبلی که بررسی کردیم، مدل طراحی شده از دسته مدل‌های یادگیری ماشین بود، با توجه به پیشرفت

^۱ kernel function

^۲ high-dimensional data

^۳ recurrent neural network

روزافزون مدل‌ها و مشاهده عملکردهای بسیار متفاوت در مدل‌های جدید که گاهی نیز بسیار بهتر از مدل‌های قدیمی است، در ادامه به سراغ مدل‌های یادگیری عمیق و شبکه‌های عصبی خواهیم رفت.

نحوه مصورسازی نیز در این مدل‌ها با گذشته متفاوت خواهد بود. در مصورسازی ماتریسی آنچه استفاده می‌شد مقادیر موجود در داده‌های نمونه بود، اما در این مدل‌ها ابرپارامترهای مدل در مصورسازی استفاده خواهند شد. یک نمونه از این‌گونه مصورسازی در مقاله [۳] آورده شده است.

بهتر است ابتدا توضیح مختصری بر شبکه RNN داشته باشیم.

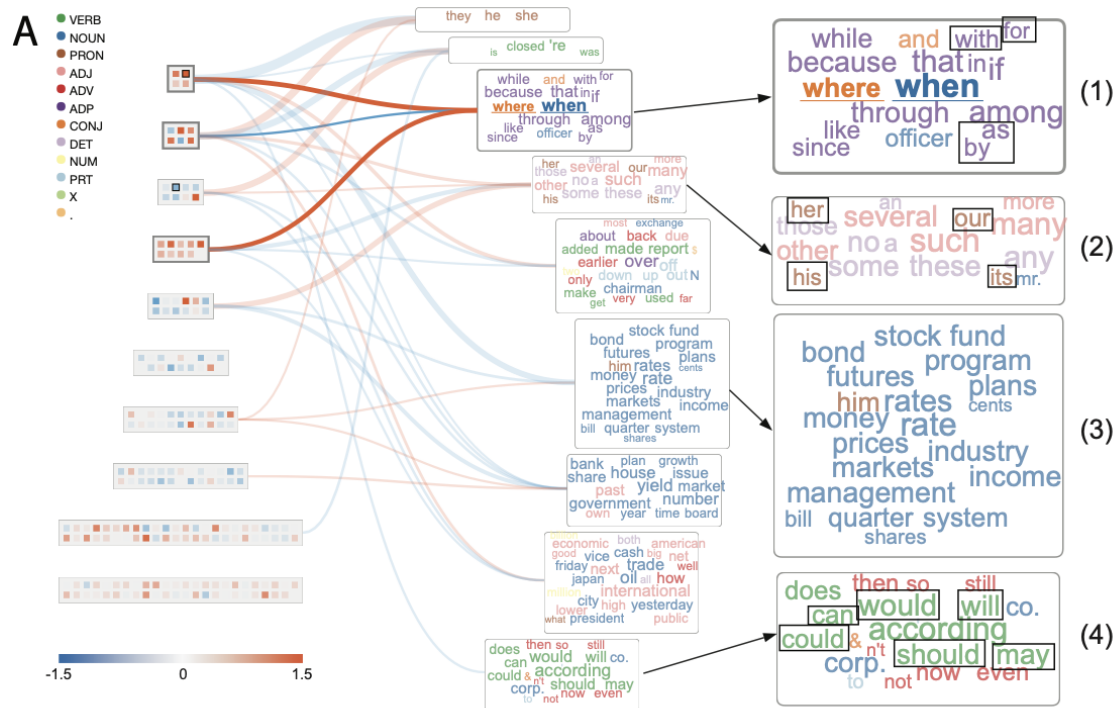
شبکه‌های عصبی بازگشتی، یک نوع از شبکه‌های عصبی مصنوعی هستند که برای پردازش داده‌های توالی و دنباله‌ای به کار می‌روند. یکی از ویژگی‌های مهم RNN این است که در هر مرحله از زمان، ورودی فعلی به همراه وضعیت^۱ قبلی، اطلاعات را پردازش می‌کند تا ارتباط‌های زمانی و متناهی را در داده‌های ورودی تشخیص دهد.

ویژگی اصلی RNN این است که از واحد بازگشتی^۲ برای نگهداری حالت و اطلاعات گذشته استفاده می‌کند. در هر مرحله زمانی، این واحد اطلاعات جدید و حالت قبلی را با هم ترکیب می‌کند و خروجی را تولید می‌کند. این خروجی همچنین به عنوان حالت برای مرحله بعدی استفاده می‌شود و به این ترتیب، ارتباط‌های زمانی در داده‌های توالی توسط RNN مدل می‌شود.

این مقاله رابطه بین حالت‌های پنهان و ورودی‌های مجزا را به عنوان نمودارهای دوبخشی فرموله می‌کند. (شکل ۵)

¹ state

² Recurrent Unit



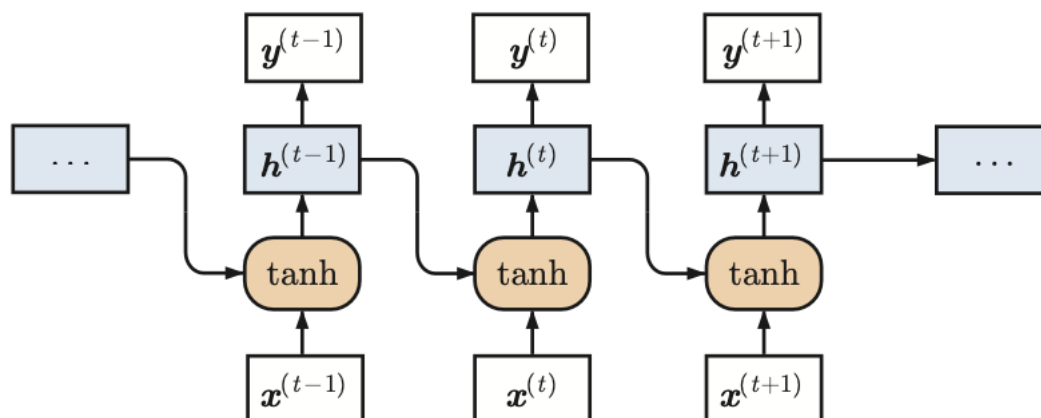
شکل ۵ گراف دوبخشی میان حالت‌های پنهان و ورودی‌های مجزا در جمله

همان‌طور که در تصویر بالا می‌بینید این مقاله کلمات را بر اساس تگ POS آنها دسته‌بندی می‌کند و رنگ متفاوت به آنها می‌دهد. به این صورت که کلمات را بر اساس تگ‌های نحوی رایج در زبان انگلیسی به دسته‌های مشابه تقسیم می‌کند. مانند: اسم (Noun) صفت (Adjective - ADJ) فعل (Verb)

به این خوشه‌بندی کلمات ابرهای کلمه^۱ و به خوشه‌بندی میان حالت‌های پنهان تراشه حافظه^۲ گفته می‌شود. حالت‌های پنهان نیز بر اساس کلمات در ورودی‌های متمایز دسته‌بندی می‌شوند. (شکل ۶)

^۱ word clouds

^۲ memory chip



شکل ۶ دسته‌بندی حالت‌های پنهان بر اساس ورودی

با این وجود، RNN دارای مشکلی به نام مشکل محو شدن یا تکرار شدن گرادیان^۱ است. این مشکل باعث می‌شود که در طی آموزش، اطلاعات در زمان بلندتر به مرور زمان از دست برود.

برای حل این مشکل و توانایی مدل‌سازی بهتر ارتباط‌های زمانی، نوعی از RNN به نام LSTM^۲ توسعه داده شده است. LSTM با استفاده از واحدهای حافظه و ساختار خاص خود، قادر به بهتر نگهداری و تعمیر اطلاعات، طی زمان بلندتر است.

RNN و نوع‌های توسعه‌یافته‌تر آن مانند LSTM، در بسیاری از برنامه‌های پردازش زبان طبیعی، تشخیص گفتار، ترجمه ماشینی، تحلیل سیگنال‌های زمانی و بسیاری دیگر از حوزه‌های مختلف مصرف دارند.

۲-۶- شبکه LSTM

در مقاله [۴] به سراغ مصورسازی حالت‌های پنهان^۳ در شبکه LSTM خواهیم رفت به همین منظور توضیح مختصری بر LSTM خواهیم داشت.

^۱ Vanishing or Exploding Gradient Problem

^۲ Long Short-Term Memory

^۳ hidden state

شبکه‌های عصبی با حافظه طولانی کوتاه‌مدت یک نوع از شبکه‌های عصبی بازگشتی به شمار می‌آیند و برای مدل‌سازی داده‌های دنباله‌ای و توالی‌های طولانی بااهمیت هستند. ویژگی اصلی LSTM این است که مشکلات مرتبط با مشکل محو شدن و تکرار شدن گرادیان در RNN را بهبود می‌بخشد.

در LSTM، واحدهای حافظه و مکانیزم‌های کنترلی مخصوصی وجود دارند که به آن اجازه می‌دهند تا طی زمان بلندتر اطلاعات را ذخیره و بازیابی کنند. این واحدها از سه بخش کلیدی تشکیل شده‌اند:

۲-۶-۱- حالت حافظه^۱

این بخش از شبکه برای ذخیره و انتقال اطلاعات طی تمام زمان‌های مختلف استفاده می‌شود. اطلاعات در این حالت به طور آهسته و پیوسته به‌مرور زمان تغییر می‌کنند.

۲-۶-۲- دروازه‌ها^۲

دارای سه دروازه اصلی است که برای کنترل جریان اطلاعات و تصمیم‌گیری‌های زمانی در شبکه استفاده می‌شوند. این دروازه‌ها شامل دروازه‌ی فراموشی^۳، دروازه‌ی ورودی^۴ و دروازه‌ی خروجی^۵ هستند.

۲-۶-۳- حالت مخفی^۶

این بخش از شبکه به‌عنوان خروجی مورد استفاده قرار می‌گیرد و اطلاعات حالت حافظه را به شکل مناسبی ترکیب می‌کند.

با استفاده از این ساختارها و مکانیزم‌ها، LSTM قادر به مدل‌سازی روابط طولی‌المدت در داده‌های توالی و دنباله‌ای می‌شود. این شبکه‌ها به‌خصوص در حوزه‌هایی مانند ترجمه ماشینی، تشخیص گفتار، تحلیل متن،

^۱ Cell State

^۲ Gates

^۳ Forget Gate

^۴ Input Gate

^۵ Output Gate

^۶ Hidden State

تولید متن و بسیاری دیگر از کاربردهای مرتبط با داده‌های زمانی مفید هستند. مقاله فوق به سراغ مصورسازی حالت‌های پنهان در لایه LSTM تعبیه شده در مدل می‌رود و برای مصورسازی از الگوریتم t -SNE^۱ در جهت کاهش بعد استفاده می‌کند. (شکل ۷)

۷-۲- الگوریتم کاهش بعد T-SNE

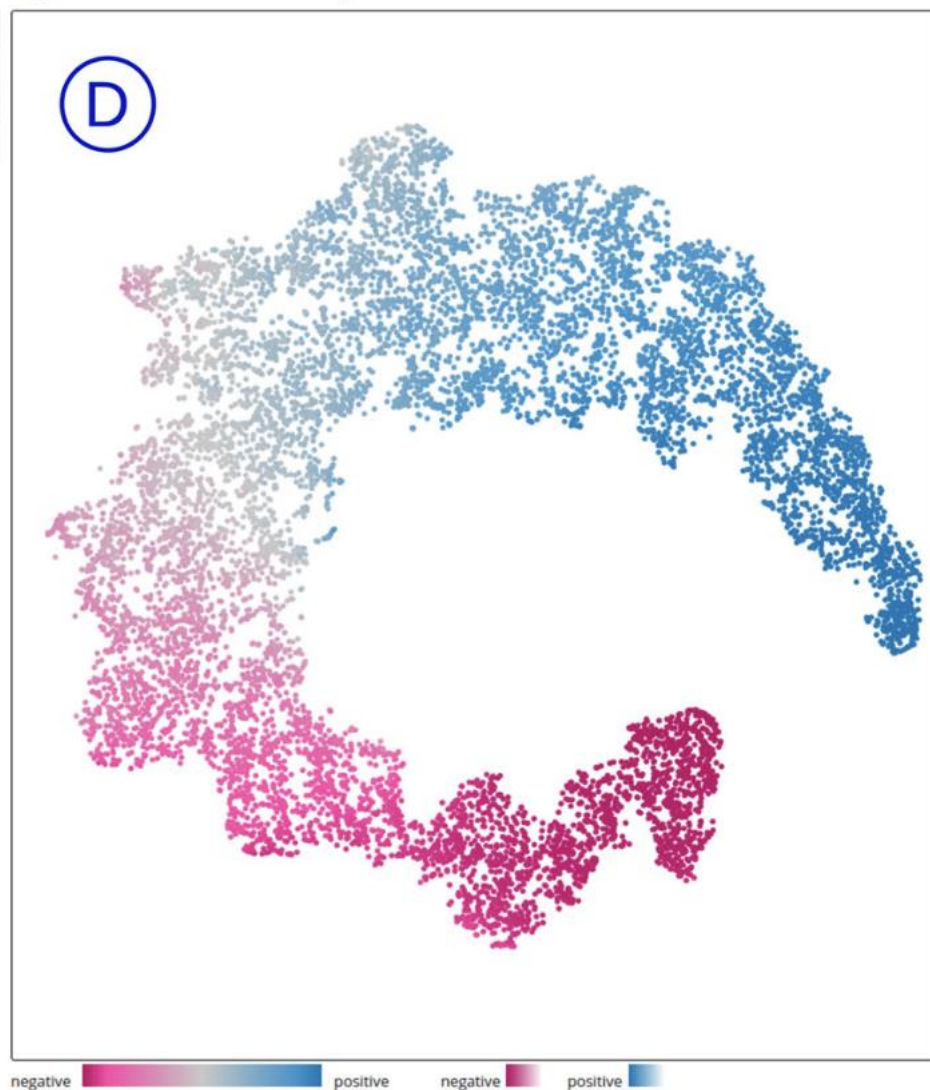
t -SNE یا توزیع مشترک توزیعی اختصاص محدود یک الگوریتم تجزیه و تحلیل داده‌ها است که برای کاهش ابعاد داده‌ها و تجزیه و تحلیل داده‌های پیچیده به کار می‌رود. این الگوریتم به صورت اصلی برای تجزیه و تحلیل داده‌های توالی یا دنباله‌ای و داده‌هایی با ساختار پیچیده مورد استفاده قرار می‌گیرد.

ویژگی اصلی t -SNE این است که ابتدا یک توزیع احتمالی در فضای ورودی (اصلی) و یک توزیع احتمالی در فضای مقصد (کاهش ابعاد شده) ایجاد می‌شود. سپس، با تلاش برای کاهش اختلاف میان این دو توزیع، داده‌ها در فضای کاهش یافته قرار می‌گیرند.

T -SNE به‌ویژه برای تجزیه و تحلیل داده‌های توالی مفید است، به‌عنوان مثال در تجزیه و تحلیل ژنوم، تجزیه و تحلیل توالی‌های زبان طبیعی، تجزیه و تحلیل داده‌های شبکه‌های اجتماعی و بسیاری دیگر. توانایی t -SNE در نمایش داده‌ها با رعایت نزدیکی‌ها و ارتباط‌های مشابه در فضای نمایشی، آن را به یک ابزار قوی برای بررسی و تجزیه و تحلیل داده‌های پیچیده تبدیل می‌کند.

^۱ t-distributed stochastic neighbor embedding

Projection of Hidden States of LSTM Layer



شکل ۷ فضای دوبعدی برای نمایش تمام حالت‌های پنهان لایه **lstm** مدل (این کاهش بعد به کمک **tsne** انجام گرفته است)

۲-۸- جمع‌بندی

در این فصل با بیان مفاهیمی چون ماتریس هیستوگرام و ماتریس شباهت برای مصورسازی آشنا شدیم. هم چنین الگوریتم‌های یادگیری ماشین همچون **svm** را مورد بررسی قرار دادیم. به دلیل نتایج متغیر و نامناسب از این روش‌ها، مفاهیم جدیدتری همچون **RNN** و **LSTM** را به همراه رویکرد جدیدی مبتنی بر مصورسازی ابر پارامترهای مدل مطرح کردیم. حال در فصل سوم به سراغ پیاده‌سازی این روش‌ها و مشاهده عملکرد آنها برای داده‌های لاگ در شبکه خواهیم رفت.

فصل سوم

شرح پروژه

۳-۱- مقدمه

در این فصل ابتدا مدل SVM را آموزش داده و برچسب داده‌های تست را توسط مدل پیش‌بینی می‌کنیم. برای درک بهتر کاربر از نتایج دریافتی مدل و تحلیل عملکرد خود مدل ماتریس شباهت را که بیانگر فاصله ویژگی‌های نمونه داده‌های تست با مرز تصمیم‌گیر مدل است، محاسبه کرده و تحلیل خواهیم کرد هر ویژگی از نمونه داده تا چه میزان بر روی برچسب پیش‌بینی‌شده تأثیرگذار است. در ادامه فصل پیاده‌سازی مدل‌های یادگیری عمیق را آغاز می‌کنیم، در این بخش مدلهایی چون MLP، RNN و LSTM پیاده‌سازی شده‌اند. نحوه مصورسازی ابر پارامترهای این مدل‌ها به کمک الگوریتم t-SNE خواهد بود.

۳-۲- پیاده‌سازی ماتریس شباهت

ابتدا دیتاست NSL-KDD را از اینترنت دانلود کرده [۵] و پیش‌پردازش‌های اولیه را روی آن انجام خواهیم داد. فایل دانلود شده حاوی دو زیرپوشه به نام‌های آموزش و تست خواهد بود.

۳-۲-۱- پیش پردازش دیتاست NSL-KDD

در گام نخست مرحله پیش پردازش ویژگی‌هایی که نوع داده‌ای آنها اسمی است را به عددی تبدیل کرده و به کمک عملیات one-hot آنها را به برداری از مقادیر ۰ و ۱ تبدیل می‌کنیم. در شکل ۸ چند نمونه از این داده‌های اسمی را مشاهده می‌کنید.

Out[]:

	protocol_type	service	flag
0	tcp	ftp_data	SF
1	udp	other	SF
2	tcp	private	S0
3	tcp	http	SF
4	tcp	http	SF

شکل ۸ داده‌های اسمی

	Protocol_type_icmp	Protocol_type_tcp	Protocol_type_udp	service_IRC	service_X11	service_Z39_50	service_aol
0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	1.0	0.0	0.0	0.0	0.0
2	0.0	1.0	0.0	0.0	0.0	0.0	0.0
3	0.0	1.0	0.0	0.0	0.0	0.0	0.0
4	0.0	1.0	0.0	0.0	0.0	0.0	0.0

شکل ۹ چند نمونه از ویژگی‌هایی که از مقادیر اسمی به عددی تبدیل شده‌اند

مرحله بعد برای پیش پردازش داده‌ها از تابع StandardScaler کتابخانه sklearn استفاده خواهیم کرد. از این تابع برای تبدیل داده‌های عددی به یک مقیاس استاندارد استفاده می‌شود تا میانگین داده‌ها صفر (۰) و انحراف معیار آنها یک (۱) باشد. این کار به کاهش تأثیر واحدهای متنوع در داده‌ها کمک می‌کند و معمولاً بهبود عملکرد مدل‌های یادگیری ماشین را به همراه دارد.

نکته حائز اهمیت در این دیتاست این است که برچسب داده‌ها نیز از جنس دسته‌بندی و اسمی است به همین منظور ما نمونه‌هایی که برچسب طبیعی^۱ خورده‌اند را برچسب ۰ و سایر داده‌ها که برچسبی از انواع حملات رخ داده در شبکه خورده‌اند را برچسب ۱ داده‌ایم. در شکل ۱۰ برچسب نهایی داده‌ها و تبدیل آنها به

^۱ normal

عدد را مشاهده می کنید.

```
labeldf=newdf['label']
# change the label column
newlabeldf=labeldf.replace({'normal': 0, 'neptune': 1, 'back': 1, 'land': 1, 'pod': 1, 'smurf': 1, 'teardrop': 1,
'ipsweep': 1, 'nmap': 1, 'portsweep': 1, 'satan': 1, 'mscan': 1, 'saint': 1,
'ftp_write': 1, 'guess_passwd': 1, 'imap': 1, 'multihop': 1, 'phf': 1, 'spy': 1, 'warezclient',
'buffer_overflow': 1, 'loadmodule': 1, 'perl': 1, 'rootkit': 1, 'ps': 1, 'sqlattack': 1, 'xterm': 1})
```

شکل ۱۰ برچسب گذاری عددی داده ها به صورت ۰ و ۱

۳-۲-۲- آموزش مدل svm

مدل svm را با هسته خطی بر روی دیتاست آموزش، آموزش می دهیم. بعد از آن مدل را به کمک کتابخانه pickle ذخیره کرده تا در آینده برای تست و ارزیابی و مصورسازی از آن استفاده کنیم. در شکل ۱۲ مراحل آموزش مدل که به کمک تابع fit از کتابخانه sklearn می باشد، به همراه ذخیره ابرپارامترهای مدل آموزش دیده را مشاهده می کنید.

۳-۲-۳- پیش بینی مدل svm

دیتاست تست را از پوشه تست فراخوانی کرده و همان عملیات های پیش پردازش که روی داده های آموزش داشتیم را مجدداً تکرار می کنیم بعد از آن مدلی را که آموزش داده بودیم و پارامترهای آن را در فایل، ذخیره کرده بودیم به کمک همان کتابخانه pickle بارگذاری می کنیم و از تابع پیش بینی^۱ برای پیش بینی مقادیر تست استفاده خواهیم کرد. در شکل ۱۱ ماتریس درهم ریختگی^۲ که اطلاعاتی از عملکرد مدل بر روی داده تست را بیان می کند، مشاهده می کنید، این ماتریس شامل مقادیری چون تعداد نمونه های واقعی مثبتی که مدل به درستی تشخیص داده است^۳ تعداد نمونه های واقعی منفی که مدل به درستی تشخیص داده است^۴ تعداد نمونه های واقعی مثبت به اشتباه به عنوان مثبت تشخیص داده است^۵ تعداد نمونه های واقعی مثبتی که مدل به اشتباه به عنوان منفی تشخیص داده است^۶

^۱ predict

^۲ Confusion Matrix

^۳ True Positives (TP)

^۴ True Negatives (TN)

^۵ False Positives (FP)

^۶ False Negatives (FN)

predict test set

```
Y_Df_pred=clf_SVM_Df.predict(X_Df_test)
pd.crosstab(Y_Df_test, Y_Df_pred, rownames=['Actual attacks'], colnames=['Predicted attacks'])
```

Predicted attacks	0	1
Actual attacks		
0	8398	1313
1	3318	9515

شکل ۱۱ پیش‌بینی داده‌های تست توسط مدل svm

SVM

```
: from sklearn.svm import SVC

clf_SVM_Df=SVC(kernel='linear', C=1.0, random_state=0)
clf_SVM_Df.fit(X_Df, Y_Df.astype(int))
```

```
▼ SVC
SVC(kernel='linear', random_state=0)
```

```
: import pickle
filename = 'finalized_model.sav'
pickle.dump(clf_SVM_Df, open(filename, 'wb'))
```

شکل ۱۲ پیش‌بینی مدل svm برای داده‌های تست

۳-۲-۴- ارزیابی مدل svm

معمولاً برای ارزیابی مدل‌های یادگیری ماشین از معیارهایی چون دقت^۱، حساسیت^۲، دقت معکوس^۳، امتیاز F۱ استفاده می‌شود و توضیح هریک به طور مختصر به صورت زیر است:

^۱ Accuracy

^۲ Recall

^۳ Precision

- دقت: نسبت تعداد نمونه‌های درست پیش‌بینی شده (TP و TN) به کل تعداد نمونه‌ها است. این معیار برای مسائل تعادلی مناسب است؛ ولی ممکن است در مواقعی که تراز توزیع کلاس‌ها متفاوت است، ناکارآمد باشد.
- حساسیت: نسبت تعداد نمونه‌های درست پیش‌بینی شده مثبت (TP) به تعداد نمونه‌های واقعی مثبت (TP + FN) است. این معیار به ما نشان می‌دهد که مدل چقدر برای تشخیص نمونه‌های مثبت کارآمد است و مهم است در مواردی که اشتباهات مثبت تأثیر مخرب دارند.
- دقت معکوس: نسبت تعداد نمونه‌های درست پیش‌بینی شده مثبت (TP) به تعداد نمونه‌های پیش‌بینی شده مثبت (TP + FP) است. این معیار به ما نشان می‌دهد که مدل چقدر دقیق در تشخیص نمونه‌های مثبت است و مهم است در مواردی که اشتباهات مثبت مخرب هستند.
- امتیاز F_1 : این امتیاز به‌عنوان معیار ترکیبی از دقت و حساسیت عمل می‌کند و میانگین هندسی این دو معیار است. معمولاً در مواردی که باید توازن بین دقت و حساسیت داشته باشیم، از امتیاز F_1 استفاده می‌شود.

برای ارزیابی مدل از ویژگی‌هایی چون دقت، حساسیت، دقت معکوس و امتیاز f_1 که در بالا هم توضیح دادیم، استفاده کرده‌ایم و نتایج دریافتی از ارزیابی مدل در (شکل ۱۳) آمده است.

Model evaluation

```
: from sklearn.model_selection import cross_val_score
from sklearn import metrics
accuracy = cross_val_score(clf_SVM_Df, X_Df_test, Y_Df_pred, cv=10, scoring='accuracy')
print("Accuracy: %0.5f (+/- %0.5f)" % (accuracy.mean(), accuracy.std() * 2))
precision = cross_val_score(clf_SVM_Df, X_Df_test, Y_Df_test, cv=10, scoring='precision')
print("Precision: %0.5f (+/- %0.5f)" % (precision.mean(), precision.std() * 2))
recall = cross_val_score(clf_SVM_Df, X_Df_test, Y_Df_test, cv=10, scoring='recall')
print("Recall: %0.5f (+/- %0.5f)" % (recall.mean(), recall.std() * 2))
f = cross_val_score(clf_SVM_Df, X_Df_test, Y_Df_test, cv=10, scoring='f1')
print("F-measure: %0.5f (+/- %0.5f)" % (f.mean(), f.std() * 2))
```

```
Accuracy: 0.99259 (+/- 0.00297)
Precision: 0.94743 (+/- 0.00726)
Recall: 0.98403 (+/- 0.00604)
F-measure: 0.96537 (+/- 0.00364)
```

شکل ۱۳ ارزیابی مدل svm

۳-۲-۵- پیاده‌سازی ماتریس شباهت

ابتدا باید داده‌های تست را به همراه برجسب‌هایی که برای آنها پیش‌بینی شده را بارگذاری کنیم. هم چنین به بارگذاری مجدد مدل نیز نیاز خواهیم داشت چرا که باید مختصات مرز تصمیم‌گیری که مدل در قسمت آموزش محاسبه کرده است را داشته باشیم.

۳-۲-۵-۱- محاسبه مقادیر ماتریس شباهت

برای محاسبه مقادیر فاصله ویژگی هر داده نمونه در دیتاست تست را با مرز تصمیم‌گیری در همان بعد در نظر گرفته می‌شود، به بیان دیگر برای ویژگی نخست مرز تصمیم‌گیر را در بعد ۰ قرار داده و فاصله منتهی ویژگی نخست در هر نمونه داده را با آن محاسبه می‌کنیم. به علت زیاد بودن ویژگی‌های این دیتاست ما ۱۲ ویژگی مهم از میان آنها را برای این کار انتخاب کرده‌ایم. در شکل ۱۴ این محاسبات را می‌بینید هم چنین فاصله منتهی ویژگی نخست ۱۰ نمونه از دیتاست تست با مرز تصمیم‌گیر در بعد ۰ چاپ شده است.

```
distance_matrix_of_sample = []
sample_size = 20
for index in range(0, len(log_dataset.columns)-1):
    feature_index = index
    # distance = |w_1 * x_1 + b| / ||w||
    w = model.coef_[0][feature_index]
    b = model.intercept_[0]
    n = w / np.linalg.norm(w)
    X_feature = np.array(log_dataset.iloc[:sample_size, feature_index]).reshape(-1, 1)
    distances = np.abs(np.dot(X_feature, w) + b) / np.linalg.norm(n)
    distance_matrix_of_sample.append(distances)
# Print the Euclidean distances
print(f"Euclidean distances based on feature {log_dataset.columns[feature_index]}:")
for i, distance in enumerate(distances):
    print(f"Sample {i+1}: {distance}")
print(distance_matrix_of_sample)
```

Euclidean distances based on feature src_bytes:

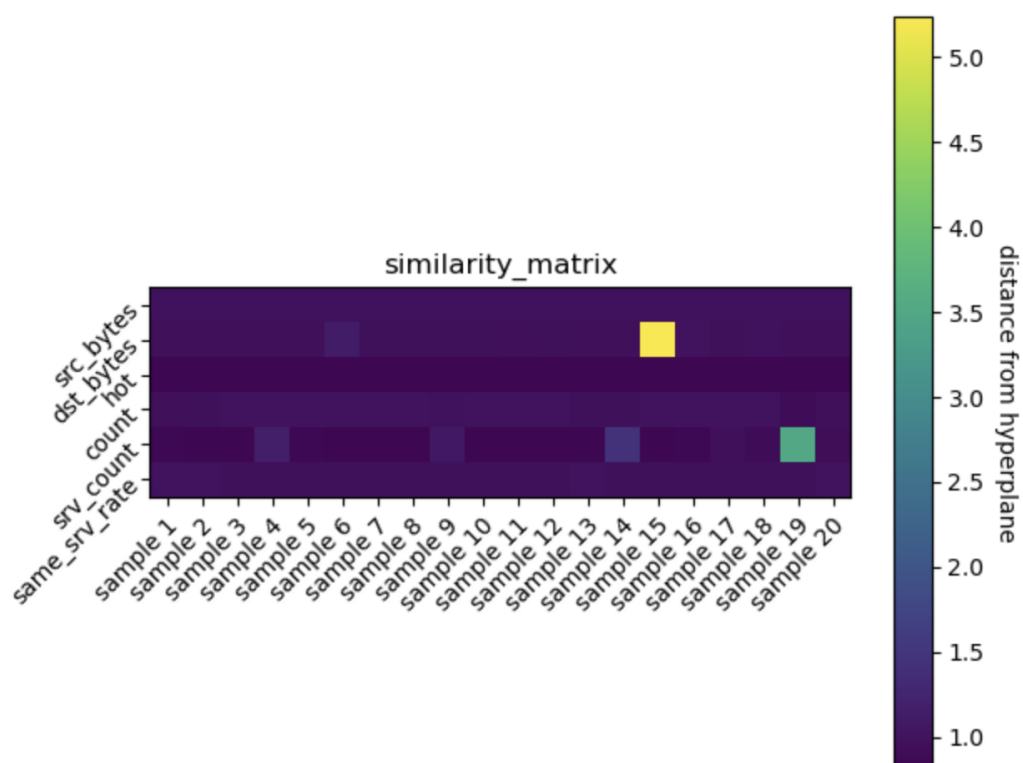
```
Sample 1: [0.98194919]
Sample 2: [0.98194919]
Sample 3: [0.98126174]
Sample 4: [0.98194813]
Sample 5: [0.98194919]
Sample 6: [0.98193505]
Sample 7: [0.98189508]
Sample 8: [0.98194236]
Sample 9: [0.98193188]
Sample 10: [0.98194781]
```

شکل ۱۴ محاسبه فاصله ۱۲ ویژگی مهم از ۱۰ نمونه از دیتاست تست با مرز تصمیم‌گیر مدل SVM

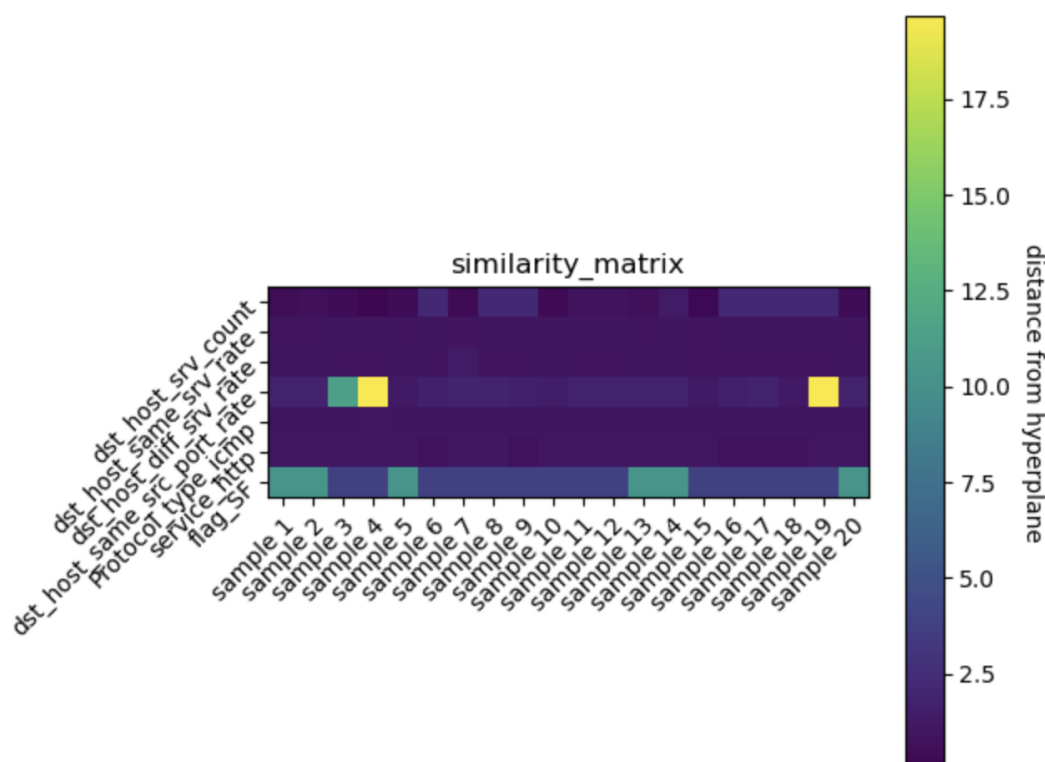
۲-۵-۲-۳- ترسیم ماتریس شباهت

در محور افقی این ماتریس برچسب مربوط به داده‌های نمونه مشاهده می‌شود مطابق (شکل ۱۵) ۲۰ داده نمونه از دیتاست تست آورده شده است.

در محور عمودی نیز ویژگی‌های موجود در دیتاست آورده شده که در (شکل ۱۵) ۶ ویژگی نخست و در (شکل ۱۶) ۶ ویژگی بعدی قرار دارند.



شکل ۱۵ ماتریس شباهت (فاصله مقادیر ویژگی‌ها برای هر داده نمونه با مرز تصمیم‌گیر در همان بعد مربوط به آن ویژگی



شکل ۱۶ ماتریس شباهت برای ۶ ویژگی بعدی

۳-۵-۲- نتیجه‌گیری و ارزیابی ماتریس شباهت

با مشاهده ماتریس‌های ترسیم شده می‌توان نتیجه گرفت که ویژگی‌هایی چون `dst_bytes` که میزان بایت‌های دریافتی در مقصد است یا `srv_count` که نشان‌دهنده اتصالات رزرو شده در مقصد است و یا `dst-host-same-srv-rate` که به معنای نسبت اتصالات رزرو شده در مقصد به کل اتصالات است از ویژگی‌هایی هستند که در برچسب پیش‌بینی‌شده توسط مدل مؤثر بوده‌اند. در واقع کاربردی که این مصورسازی می‌تواند برای کاربر داشته باشد بدین‌گونه است که کاربر می‌تواند در صورت عدم دریافت نتیجه مناسب از برچسب پیش‌بینی‌شده توسط مدل به سراغ بررسی این ویژگی‌ها برود.

۳-۳- پیاده‌سازی مدل‌های یادگیری عمیق

ساختار پروژه بدین صورت است که مدل از دو زیر مدل تشکیل شده است:

۳-۳-۱- زیر مدل مربوط به مصورسازی

این مدل از چندین لایه تشکیل شده است که لایه نخست آن embedding می‌باشد و در تمام حالات ثابت است، لایه دوم و سوم متغیر است و بسته به حالت می‌تواند لایه dense برای مدل mlp یا لایه LSTM برای مدل LSTM باشد. در پایان روی لایه آخر این مدل تابع predict برای ۱۰۰ نمونه از دیتاست تست فراخوانی می‌شود و مقادیر خروجی آن برای مصورسازی به t-sne داده می‌شود. هریک از حالات فوق در ادامه توضیح داده خواهند شد.

۳-۳-۲- زیر مدل مربوط به پیش‌بینی

لایه‌های این زیر مدل در تمام حالات ثابت است و به‌صورت زیر است:

```
summary of model_pred
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
dropout_1 (Dropout)	(None, 100)	0
dense_4 (Dense)	(None, 1)	101

```

Total params: 101 (404.00 Byte)
Trainable params: 101 (404.00 Byte)
```

شکل ۱۷ زیرمدل مربوط به پیش‌بینی مدل

خروجی لایه آخر این مدل برچسب نهایی مربوط به هر داده نمونه را تعیین می‌کند. از این برچسب تنها برای تعیین رنگ هر کلمه در فضای مصورسازی استفاده شده است. مدل‌های زیر تنها در زیر مدل اول متفاوت هستند. هم چنین به جهت بررسی و درک بهتر عملکرد این مدل‌ها از دیتاست مربوط به نظرات کاربران برای فیلم در سایت imdb استفاده شده است و در آخر برای بهترین مدل که مصورسازی کاربردی‌تری دارد، از دیتاست zookeeper که مربوط به لاگ است، استفاده شده است.

علت استفاده از دیتاست نظرات فیلم به جای متن لاگ به این جهت است که متن لاگ حاوی کلمات گاهی تخصصی و فاقد معنا می‌باشد و نمی‌توان به خوبی مصورسازی این گونه کلمات را مورد نقد قرار داد.

۳-۳-۳- مصورسازی با t-sne

همان‌طور که پیش‌تر هم گفته شد برای مصورسازی از الگوریتم t-sne استفاده شده است در واقع t-sne کاهش بعد را برای نمایش داده‌ها انجام می‌دهد، خروجی زیر مدل اول بسته به نوع لایه‌ای که استفاده شده متفاوت است؛ اما در هر صورت این خروجی بیش از دو بعد می‌باشد و برای نمایش در فضای دوبعدی ما به یک الگوریتم کاهش بعد همچون t-sne نیاز خواهیم داشت. برای این منظور سایر الگوریتم‌های مشابه مانند pca نیز بررسی شدند؛ اما به جهت دریافت نتیجه مشابه و کارکرد ساده‌تر t-sne از t-sne استفاده شده است.

با مطالعه سازوکار الگوریتم t-sne، کارکرد دیگر آن می‌تواند همچون الگوریتم‌های انتخاب ویژگی^۱ باشد چرا که کاهش بعد به صورتی است که ابعاد با اهمیت بالاتر را نگه خواهد داشت. به طور مثال در (شکل ۱۸) ابعاد ماتریس خروجی از لایه rnn (۱۰۰, ۱۰۰۰۰) است که نشان‌دهنده سائز ماتریس حالت‌های پنهان است با حذف مقادیر padding این سائز به (۱۰۰, ۹۷۴۵) خواهد رسید. t-sne با کاهش بعدی که انجام می‌دهد این ماتریس را به فضای دوبعدی می‌برد (۲, ۹۷۴۵) پیشینه سائزی که برای هر ورودی در نظر گرفته شده ۱۰۰ می‌باشد و اگر داده ای کمتر از ۱۰۰ کلمه یا به اصطلاح توکن داشته باشد با اعمال padding سائز آن را به ۱۰۰ می‌رسانیم.

```

PROBLEMS  DEBUG CONSOLE  TERMINAL  GITBINS  SQL CONSOLE MESSAGES  OUTPUT
347/352 [=====] - ETA: 0s - loss: 0.1715 - accuracy: 0.93
350/352 [=====] - ETA: 0s - loss: 0.1712 - accuracy: 0.93
352/352 [=====] - 10s 27ms/step - loss: 0.1712 - accuracy
: 0.9348 - val_loss: 0.5257 - val_accuracy: 0.8144
782/782 - 3s - loss: 0.5163 - accuracy: 0.8040 - 3s/epoch - 4ms/step
Accuracy: 80.40%
4/4 [=====] - 0s 4ms/step
hidden state shape :
(10000, 100)
313/313 [=====] - 0s 338us/step
4/4 [=====] - 0s 4ms/step
hs_nopad shape :
(9745, 100)
[t-SNE] Computing 301 nearest neighbors...
[t-SNE] Indexed 9745 samples in 0.001s...
[t-SNE] Computed neighbors for 9745 samples in 1.048s...
[t-SNE] Computed conditional probabilities for sample 1000 / 9745
[t-SNE] Computed conditional probabilities for sample 2000 / 9745
[t-SNE] Computed conditional probabilities for sample 3000 / 9745
[t-SNE] Computed conditional probabilities for sample 4000 / 9745
[t-SNE] Computed conditional probabilities for sample 5000 / 9745
[t-SNE] Computed conditional probabilities for sample 6000 / 9745
[t-SNE] Computed conditional probabilities for sample 7000 / 9745
[t-SNE] Computed conditional probabilities for sample 8000 / 9745
[t-SNE] Computed conditional probabilities for sample 9000 / 9745
[t-SNE] Computed conditional probabilities for sample 9745 / 9745
[t-SNE] Mean sigma: 0.364542
[t-SNE] KL divergence after 250 iterations with early exaggeration: 72.704544
[t-SNE] KL divergence after 1000 iterations: 2.151699
projection_nopad shape :
(9745, 2)
~/De/ft/D/hiddenStatesVis/dataPreparation on master !6 718

```

شکل ۱۸ کاهش بعد توسط tsn-e برای مصورسازی حالت‌های پنهان

^۱ feature selection

۳-۴- مدل mlp

برای این مدل ابتدا از یک لایه embedding برای تبدیل تمامی لغات استفاده شده در دیتاست به مجموعه‌ای از اعداد اعشاری استفاده شده است.

از یک لایه dense بعد از embedding و یک لایه flatten بعد از آن استفاده شده به این علت که برای مصورسازی خروجی این لایه به t-sne داده می‌شود و ورودی‌ای که t-sne می‌گیرد دوبعدی است که بعد اول شامل تعداد داده‌ی نمونه است و بعد دوم بیشینه توکنی است که یک داده نمونه می‌تواند داشته باشد.

۳-۴-۱- آموزش و پیش‌بینی مدل

همان‌طور که پیش‌تر گفته شد به دو زیر مدل نیاز خواهیم داشت دو لایه نخست زیر مدل مربوط به مصورسازی خواهند بود و دو لایه آخر برای پیش‌بینی استفاده می‌شوند. ساختار کل مدل در (شکل ۱۹) آورده شده است.

```
python3 model_summary.py
Random seed: 99
hello1
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 100, 32)	2834784
dense (Dense)	(None, 100, 1)	33
flatten (Flatten)	(None, 100)	0
dropout (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 1)	101

شکل ۱۹ ساختار کلی مدل mlp

این مدل در یک epoch آموزش داده شده است و دقت آن در (شکل ۲۰) آورده شده است.

```
None
hello2
352/352 [=====] - 2s 5ms/step - loss: 0.6582 - accuracy: 0.5914 - val_loss: 0.5552 - val_accuracy: 0.7616
782/782 - 0s - loss: 0.5548 - accuracy: 0.7538 - 447ms/epoch - 572us/step
Accuracy: 75.38%
Summary of model_lstm
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
--------------	--------------	---------

شکل ۲۰ دقت مدل mlp

۳-۴-۲- مصورسازی مدل mlp

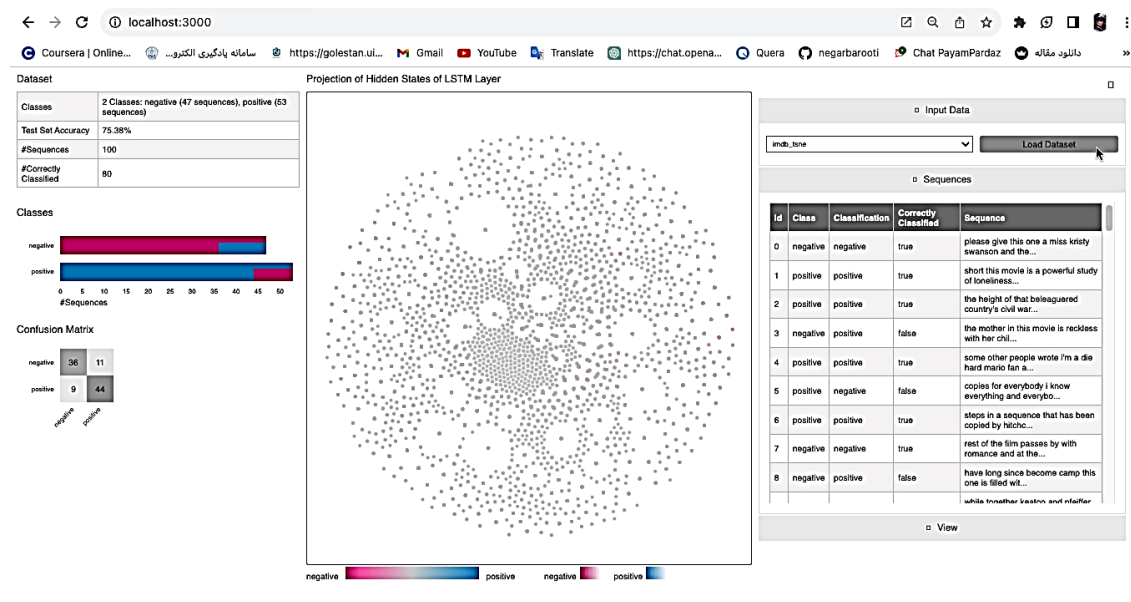
با اینکه دقت مدل تقریباً بالاست اما مصورسازی آن کاربردی برای کاربر ندارد و تمام کلمات موجود در ۱۰۰ نمونه نخست دیتاست تست به صورت بهم‌ریخته و بدون هیچ‌گونه نظم یا ساختار مشخصی در صفحه قرار گرفته است. در واقع می‌توان نتیجه گرفت که آموزش مدل و برچسب پیش‌بینی شده توسط آن تأثیری در

مصورسازی نداشته است.

از این جهت که لایه dense نمی‌تواند مانند سایر لایه‌ها (LSTM, RNN) اطلاعات قبلی را در داده‌های ترتیبی نگه دارد.

همان‌طور که در شکل ۲۱ نتیجه این مصورسازی را می‌بینید هر نقطه در مستطیل بزرگ وسط تصویر معادل یک کلمه در نمونه داده است و این تصویر شامل ۱۰۰ نمونه از دیتاست تست است.

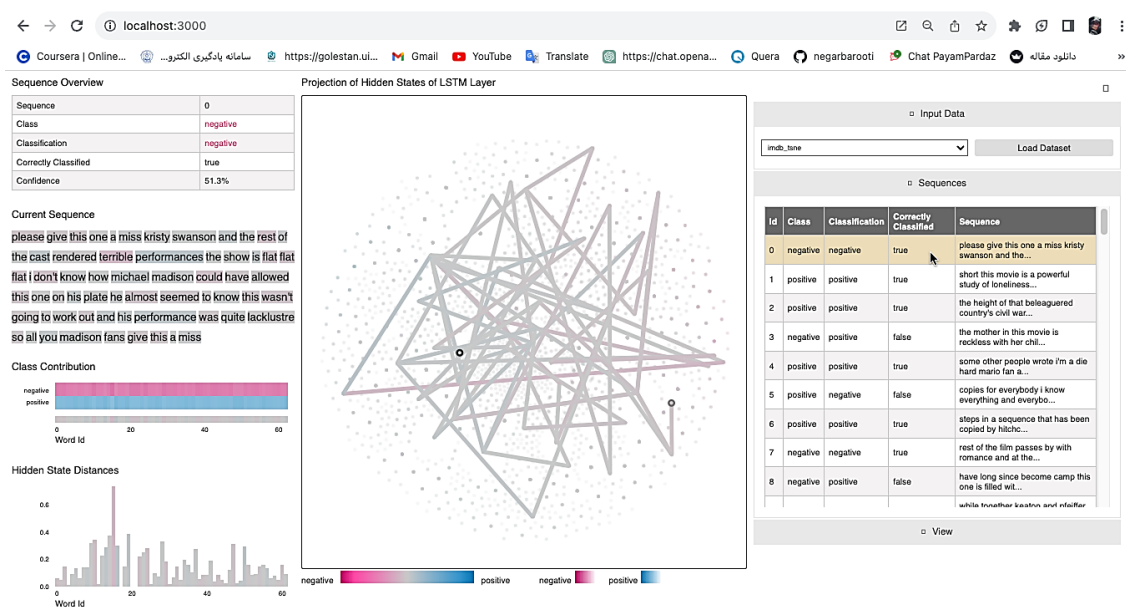
نحوه این مصورسازی بدین‌گونه است که برای هر نمونه ۱۰۰ کلمه به‌صورت بیشینه در نظر گرفته می‌شود در نتیجه خروجی لایه flatten نیز تعداد نمونه در ۱۰۰ می‌باشد و با اعمال این ورودی ۱۰۰ در این لایه، به خروجی (۱۰۰ و ۱۰۰ و ۱۰۰) خواهیم رسید که اگر دو بعد اول را به‌صورت عمودی ترکیب کنیم به آرایه‌ای با ابعاد ۱۰۰۰۰ در ۱۰۰ خواهیم رسید و با حذف padding سائز آرایه نهایی (۱۰۰، ۹۷۴۵) خواهد بود که شامل ۹۷۴۵ نقطه‌ای است که t-sne آنها را در فضای دوبعدی شکل ۲۱ ترسیم کرده است.



شکل ۲۱ نتیجه مصورسازی ابرپارامترهای مدل mlp

۳-۴-۳- نتایج مدل mlp

مدل mlp در نمایش رنگ که همان برچسب پیش‌بینی شده توسط مدل است نیز نتایج نامناسبی ارائه کرده است؛ زیرا همان‌طور که در (شکل ۲۲) نیز می‌بینید رنگ تمام کلمات در یک داده نمونه با برچسب منفی خاکستری است در واقع این به معنای آن است که مدل نتوانسته به‌صورت ترتیبی داده‌ها را آموزش ببیند و اطلاعات قبلی در یک نمونه را نگه دارد یا از آن برای پیش‌بینی برچسب نهایی استفاده کند.



شکل ۲۲ مصورسازی یک داده نمونه با mlp و نمایش نامناسب رنگ برای کلمات ورودی

۳-۵- مدل RNN

در این حالت از همان ساختار مدل قبلی استفاده شده است با این تفاوت که به جای لایه‌های dense و flatten از یک لایه rnn استفاده شده است.

در واقع زیر مدل مربوط به مصورسازی تنها شامل لایه‌های embedding و rnn است و خروجی لایه rnn که همان ابرپارامترها یا به اصطلاح وزن‌های این لایه است، مصورسازی می‌شود.

۳-۵-۱- آموزش و پیش‌بینی مدل

ساختار مدل در این حالت به صورت زیر است. هم چنین نتایج به دست آمده از مدل نیز قابل مشاهده است. در (شکل ۲۳) نیز می‌توان مشاهده کرد که دقت مدل نسبت به مدل mlp افزایش یافته است. دقت مدل در حالتی که تنها از یک لایه dense استفاده شده بود ۷۵٪ بود و در این حالت ۸۰٪ می‌باشد.

```

28 def createModel(x_train, lstmUnits, outputUnits, activation):
29     # Instantiate the layers of the model.
30     embedd = Embedding(input_dim=(np.max(x_train) + 1), output_dim=32, input_length=x_train.sha
31     # lstm_1 = LSTM(units=lstmUnits)
32     rnn_1 = SimpleRNN(units=lstmUnits)
33     drop_2 = Dropout(rate=0.5)
34     output = Dense(units=outputUnits, activation=activation)
35
36     model = Sequential()
37     model.add(embedd)

```

PROBLEMS DEBUG CONSOLE TERMINAL GITLENS SQL CONSOLE: MESSAGES OUTPUT

```

347/352 [=====] - ETA: 0s - loss: 0.1715 - accuracy: 0.93
350/352 [=====] - ETA: 0s - loss: 0.1712 - accuracy: 0.93
352/352 [=====] - 10s 27ms/step - loss: 0.1712 - accuracy
: 0.9348 - val_loss: 0.5257 - val_accuracy: 0.8144
782/782 - 3s - loss: 0.5163 - accuracy: 0.8040 - 3s/epoch - 4ms/step
Accuracy: 80.40%
4/4 [=====] - 0s 4ms/step
hidden state shape :
(10000, 100)
313/313 [=====] - 0s 338us/step
4/4 [=====] - 0s 4ms/step
hs_nopad shape :
(9745, 100)
[t-SNE] Computing 301 nearest neighbors...
[t-SNE] Indexed 9745 samples in 0.001s...
[t-SNE] Computed neighbors for 9745 samples in 1.048s...
[t-SNE] Computed conditional probabilities for sample 1000 / 9745
[t-SNE] Computed conditional probabilities for sample 2000 / 9745
[t-SNE] Computed conditional probabilities for sample 3000 / 9745
[t-SNE] Computed conditional probabilities for sample 4000 / 9745
[t-SNE] Computed conditional probabilities for sample 5000 / 9745
[t-SNE] Computed conditional probabilities for sample 6000 / 9745
[t-SNE] Computed conditional probabilities for sample 7000 / 9745
[t-SNE] Computed conditional probabilities for sample 8000 / 9745
[t-SNE] Computed conditional probabilities for sample 9000 / 9745
[t-SNE] Computed conditional probabilities for sample 9745 / 9745
[t-SNE] Mean sigma: 0.364542
[t-SNE] KL divergence after 250 iterations with early exaggeration: 72.704544
[t-SNE] KL divergence after 1000 iterations: 2.151699
projection_nopad shape :
(9745, 2)

```

zsh dataPreparation
zsh visualAnalytics

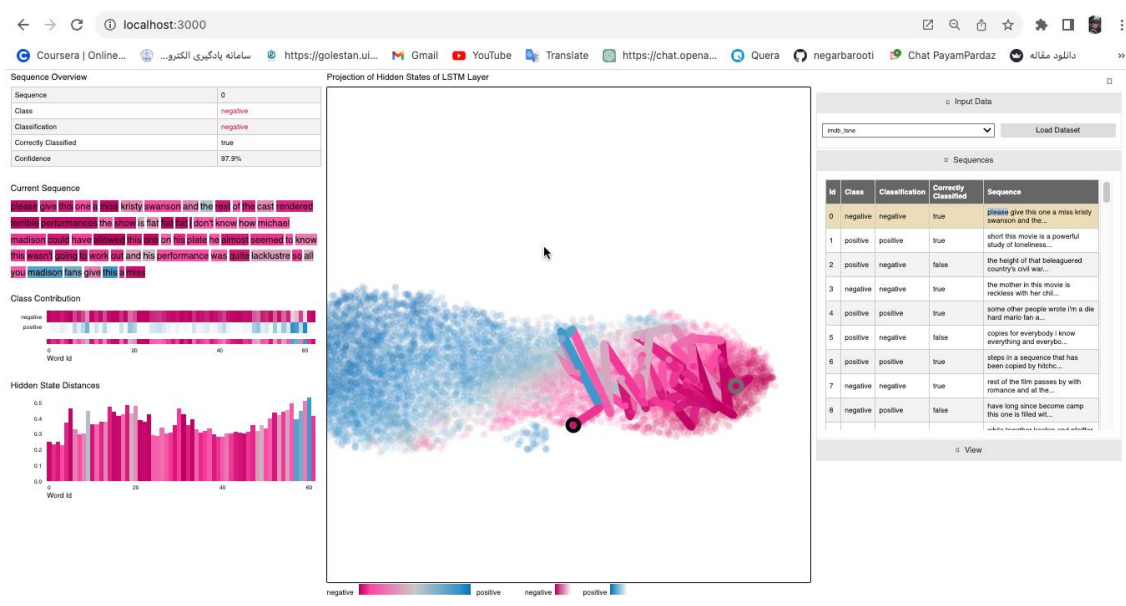
شکل ۲۳ آموزش با مدل rnn

۳-۵-۲- مصورسازی مدل RNN

پس از مصورسازی ابرپارامترهای لایه rnn دیدیم که نتایج مصورسازی نیز بسیار بهتر از حالت mlp است در واقع داده با ساختار و الگوی منطقی‌تری در صفحه‌نمایش داده می‌شود و بر اساس برچسب پیش‌بینی‌شده به دو نیمکره در سمت‌های چپ و راست تقسیم شده و قسمت میانی در واقع شامل کلماتی است که در پیش‌بینی مدل تأثیرگذار نبوده‌اند و یا به اصطلاح خنثی هستند. کلماتی که رنگ آنها در مصورسازی نیز خاکستری است. در شکل ۲۴ نمونه این مصورسازی را برای یک داده با برچسب منفی می‌بینید و در شکل ۲۵ نمونه داده با برچسب مثبت مصورسازی شده است.

در سمت چپ تصویر نیز مصورسازی برچسب پیش‌بینی‌شده توسط مدل به کمک رنگ صورت گرفته است.

نحوه این مصورسازی برگرفته از کتابخانه captum است. برای تعیین رنگ هر کلمه در یک نمونه داده، باید نمونه داده موردنظر را تا آن کلمه مشخص قطع کرد و به تابع پیش‌بینی مدل داد. برچسب پیش‌بینی‌شده توسط مدل برای آن زیر قسمت از نمونه متعلق به آخرین کلمه در زیرنمونه خواهد بود و تعیین‌کننده رنگ آن کلمه می‌باشد.

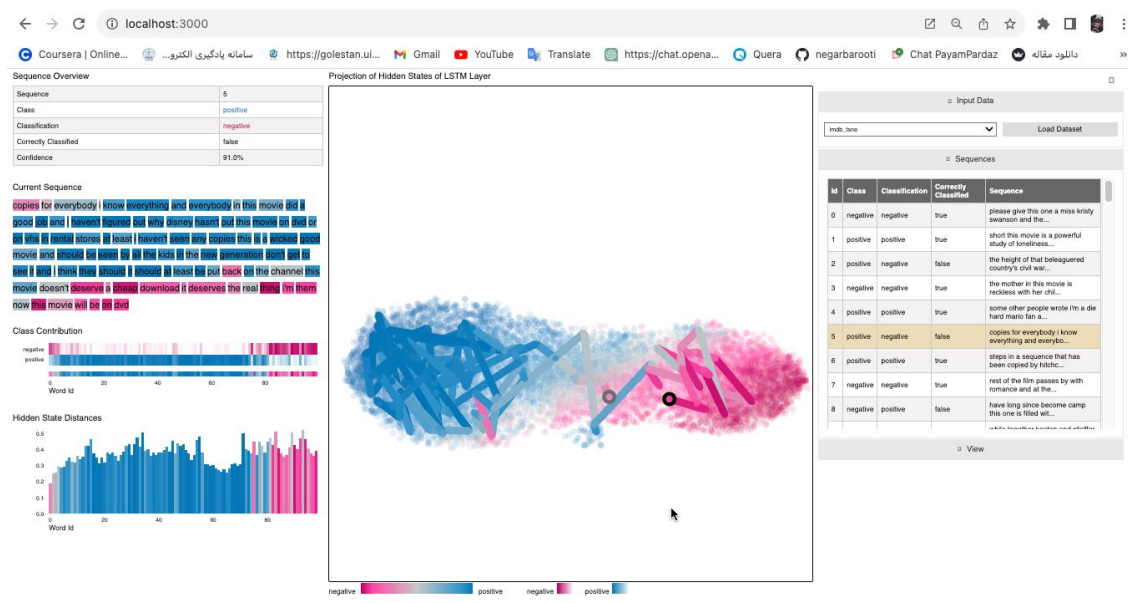


شکل ۲۴ مصورسازی یک داده نمونه با برچسب منفی با مدل rnn

۳-۵-۳- نتایج مدل RNN

نتیجه‌ای که از این مدل دریافت می‌شود برای مصورسازی و پیش‌بینی بسیار مناسب است تنها عیبی که می‌تواند داشته باشد هم‌پوشانی کلمات در نمایش داده‌ها است که به علت نزدیک بودن مقادیر ابرپارامترها در لایه rnn به یکدیگر است.

با در نظر داشتن ۱۰۰ سلول در لایه در لایه rnn در واقع برای هر کلمه ۱۰۰ حالت پنهان داریم و چون تعداد کلمات بدون در نظر گرفتن تکراری بودن یا نبودن آنها در بیشترین حالت ۱۰۰۰۰ می‌باشد. (۱۰۰ داده نمونه که هر کدام بیشینه ۱۰۰ توکن دارند) در واقع سائز ماتریس (۱۰۰، ۱۰۰۰۰) خواهد بود. در پایان برای مصورسازی t-sne این ۱۰۰ حالت پنهان که برای هر توکن داریم را به دو حالت پنهان کاهش می‌دهد تا بتوان این نقاط را در فضای دو بعد ترسیم کرد.



شکل ۲۵ مصورسازی با مدل rnn برای یک داده نمونه با برچسب مثبت

۳-۶- LSTM مدل

در این حالت از همان ساختار مدل قبلی استفاده شده است با این تفاوت که به جای لایه‌های dense و flatten از یک لایه lstm استفاده شده است.

در واقع زیر مدل مربوط به مصورسازی تنها شامل لایه‌های embedding و lstm است و خروجی لایه lstm که همان ابرپارامترها یا به اصطلاح وزن‌های این لایه است، مصورسازی می‌شود.

۳-۶-۱- آموزش و پیش‌بینی مدل

ساختار کلی مدل در شکل ۲۶ آورده شده است. هم چنین نتایج حاصل از آموزش و پیش‌بینی مدل در شکل ۲۷ قرار دارد که نشان می‌دهد دقت مدل نسبت به حالت قبل بازهم افزایش داشته و به ۸۴٪ رسیده است. البته باید در نظر داشت که تعداد epoch نیز به ۵ افزایش یافته و این مورد در افزایش دقت مدل بی‌تأثیر نخواهد بود.

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 100, 32)	2834784
lstm (LSTM)	(None, 100)	53200
dropout (Dropout)	(None, 100)	0
dense (Dense)	(None, 1)	101
Total params: 2888085 (11.02 MB)		
Trainable params: 2888085 (11.02 MB)		
Non-trainable params: 0 (0.00 Byte)		

شکل ۲۶ ساختار مدل lstm

```

27
28 def createModel(x_train, lstmUnits, outputUnits, activation):
29     # Instantiate the layers of the model.
30     embedd = Embedding(input_dim=(np.max(x_train) + 1), output_dim=32, input_length=x_train.shape[1])
31     lstm_1 = LSTM(units=lstmUnits)
32     drop_2 = Dropout(rate=0.5)
33     output = Dense(units=outputUnits, activation=activation)
34
35     model = Sequential()
36     model.add(embedd)
37     model.add(lstm_1)
38     model.add(drop_2)

```

PROBLEMS DEBUG CONSOLE TERMINAL GITLENS SQL CONSOLE OUTPUT

`/De/f1/deepLearningVis/hiddenStatesVis/dataPreparation` on master 14 73 ----- took 4m 3s `hiddenStatesVis.py` | base.py at 11:17:11

```

> python3 indblstm.py
Random seed: 99
hello
Epoch 1/5
352/352 [=====] - 37s 101ms/step - loss: 0.4822 - accuracy: 0.7696 - val_loss: 0.3553 - val_accuracy: 0.8476
Epoch 2/5
352/352 [=====] - 38s 108ms/step - loss: 0.2809 - accuracy: 0.8901 - val_loss: 0.3367 - val_accuracy: 0.8596
Epoch 3/5
352/352 [=====] - 47s 134ms/step - loss: 0.2252 - accuracy: 0.9165 - val_loss: 0.3426 - val_accuracy: 0.8572
Epoch 4/5
352/352 [=====] - 32s 91ms/step - loss: 0.1893 - accuracy: 0.9304 - val_loss: 0.4362 - val_accuracy: 0.8524
Epoch 5/5
352/352 [=====] - 28s 81ms/step - loss: 0.1593 - accuracy: 0.9415 - val_loss: 0.3597 - val_accuracy: 0.8500
782/782 - 11s - loss: 0.3719 - accuracy: 0.8474 - 11s/epoch - 14ms/step
Accuracy: 84.74%
4/4 [=====] - 0s 13ms/step
313/313 [=====] - 0s 379us/step
4/4 [=====] - 0s 13ms/step
[t-SNE] Computing 301 nearest neighbors...
[t-SNE] Indexed 9745 samples in 0.000s...
[t-SNE] Computed neighbors for 9745 samples in 0.971s...
[t-SNE] Computed conditional probabilities for sample 1000 / 9745
[t-SNE] Computed conditional probabilities for sample 2000 / 9745
[t-SNE] Computed conditional probabilities for sample 3000 / 9745
[t-SNE] Computed conditional probabilities for sample 4000 / 9745
[t-SNE] Computed conditional probabilities for sample 5000 / 9745
[t-SNE] Computed conditional probabilities for sample 6000 / 9745
[t-SNE] Computed conditional probabilities for sample 7000 / 9745
[t-SNE] Computed conditional probabilities for sample 8000 / 9745
[t-SNE] Computed conditional probabilities for sample 9000 / 9745
[t-SNE] Computed conditional probabilities for sample 9745 / 9745
[t-SNE] Mean sigma: 0.088018
[t-SNE] KL divergence after 250 iterations with early exaggeration: 64.189743
[t-SNE] KL divergence after 1000 iterations: 0.878966

```

شکل ۲۷ آموزش و پیش‌بینی مدل lstm

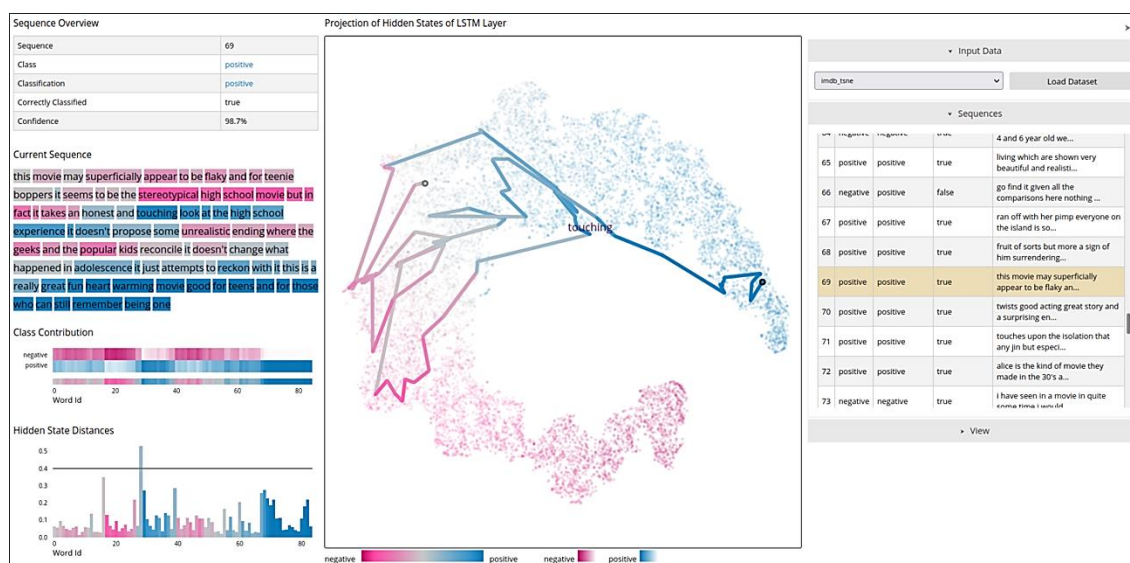
۳-۶-۲- مصورسازی مدل LSTM

مصورسازی مدل در این حالت مانند دو حالت قبلی خواهد بود و هیچ‌گونه تغییری در ابعاد ماتریس‌ها نداشتیم، با این‌وجود نحوه قرارگیری نقاط در فضای دوبعدی محاسبه شده توسط t-sne متفاوت از حالات قبل می‌باشد.

۳-۶-۳- نتایج مدل LSTM

نتیجه نهایی این مصورسازی در شکل ۲۸ آورده شده است و همان طور که ملاحظه می کنید تصویر نهایی به خوبی کلمات را بر اساس تأثیرگذاری آنها در برچسب نهایی پیش بینی شده به دو دسته تقسیم کرده یا به اصطلاح رنگ های مجزا، جدا از هم قرار گرفته اند.

هم چنین در سمت چپ تصویر نیز کلمات به همراه رنگ آنها در یک نمونه آورده شده است که شدت این رنگ بسته به میزان درجه اطمینان مدل از برچسب پیش بینی شده متفاوت خواهد بود.



شکل ۲۸ مصورسازی نهایی مدل lstm

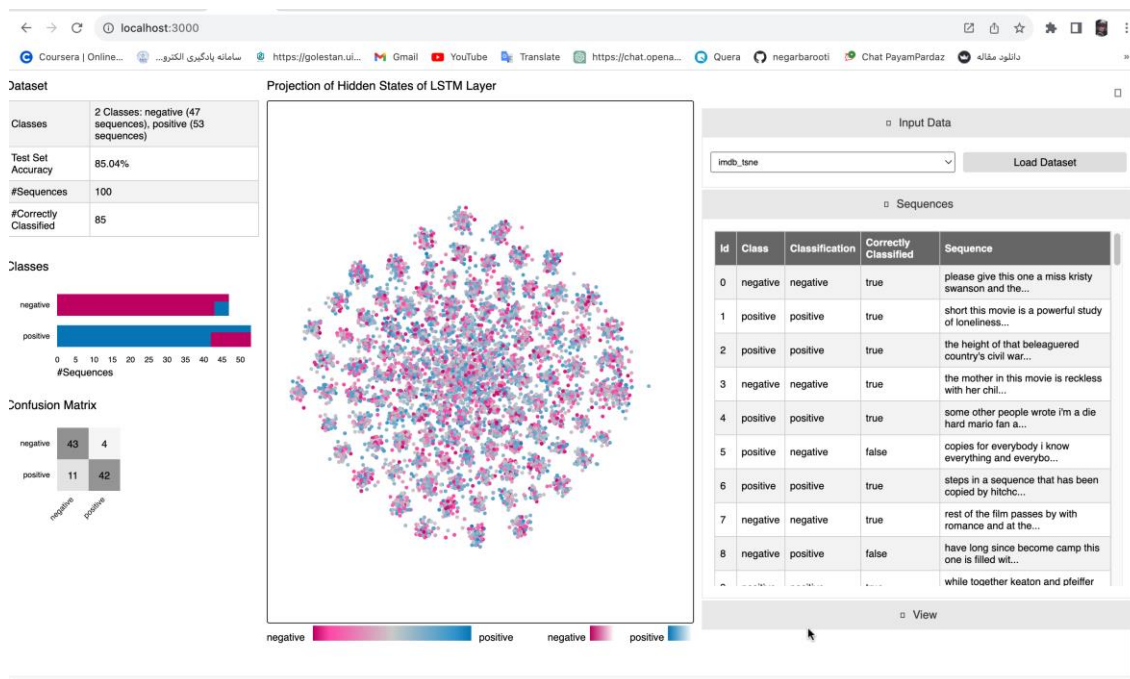
۳-۷- ارزیابی مدل های یادگیری عمیق

برای ارزیابی بهتر مدل های یادگیری عمیق به سراغ مصورسازی کلمات موجود در لاگ پیش از ورود به لایه های مدل، رفته ایم. همان طور که در شکل ۲۹ نیز می بینید دسته بندی کلمات بر اساس رنگ صورت نگرفته است و کلمات بدون هیچ الگوی مشخص و کاربردی برای کاربر، در صفحه قرار گرفته اند.

در قسمت های قبل مصورسازی ابر پارامترهای مدل را نیز دیده بودید. اکنون می توان به خوبی عملکرد مدل های یادگیری عمیق را ارزیابی کرد.

جداسازی کلمات بر اساس برچسب های آنها برای کاربر حائز اهمیت است و مصورسازی دیتای خام به کاربر کمکی نخواهد کرد، در نتیجه استفاده از مدل های یادگیری عمیق در فرایند مصورسازی بسیار مفید خواهد بود.

شاید بتوان برای پیش‌بینی مدل از سایر مدل‌ها یا حتی مدل‌های یادگیری ماشین استفاده کرد؛ اما برای مصورسازی بهتر است از شبکه‌های عصبی و لایه‌های خاص آن استفاده کرد.



شکل ۲۹ مصورسازی دیتاست آموزش به کمک tsne-e

۳-۸- پیاده‌سازی بهترین مدل برای مصورسازی بر روی دیتاست لاگ

باتوجه به مدل‌های آموزش داده شده و مصورسازی‌های صورت گرفته روی ابرپارامترهای آنها بدین نتیجه رسیدیم که مدل lstm دقت بیشتری دارد، هرچند که از لحاظ مصورسازی مدل‌های lstm و rnn هر دو به خوبی کلمه‌های موجود در ۱۰۰ نمونه از دیتاست تست را نمایش می‌دهند. در همین راستا دیتاست zookeeper [۶] که مربوط به لاگ است را برای آموزش به این مدل می‌دهیم.

۳-۸-۱- دیتاست zookeeper

در واقع ZooKeeper یک سرویس متمرکز برای حفظ اطلاعات پیکربندی، نام‌گذاری، ارائه همگام‌سازی توزیع شده و ارائه خدمات گروهی است.

این دیتاست شامل مجموعه‌ای از لاگ‌های این سرویس در یک دوره ۲۶ روزه است.

نکته حائز اهمیت در زمان آموزش مدل‌ها بر روی داده‌ی لاگ این است که لاگ شامل چندین ستون ویژگی با اطلاعات کم‌اهمیت برای مدل است که نیازی به آموزش توسط مدل ندارند از جمله تاریخ و زمان وقوع لاگ و یا نود مرتبط با لاگ در شبکه. البته ممکن است در آینده و پس از دریافت نتیجه مدل، کاربر بخواهد سایر این ویژگی‌ها را برای داده‌ی ناهنجار یافته شده، بداند؛ اما فعلاً برای یافتن داده‌ی ناهنجار و هنگام استفاده از مدل‌های پردازش متن به این ویژگی‌ها نیازی نخواهیم داشت.

دو ویژگی به صورت متن و از جنس داده ترتیبی در این دیتاست وجود دارد که در شکل ۳۰ این ویژگی را برای چند نمونه از لاگ‌ها خواهید دید.

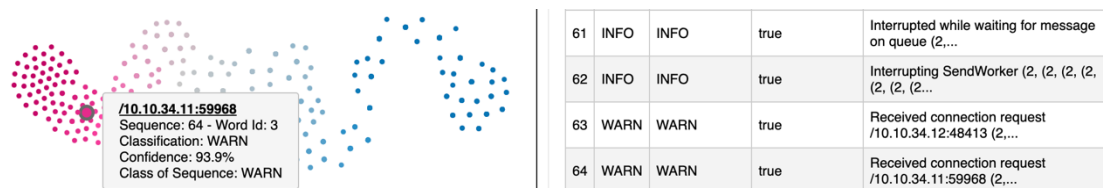
- محتوای اصلی رویداد: در این ویژگی متن اصلی لاگ قرار گرفته است.
- قالب رویداد : لاگ‌ها بر اساس ساختار به چند دسته تقسیم می‌شوند و هر دسته عدد متمایزی دریافت می‌کند که ۴۲ventide نام دارد . در این ویژگی ساختار کلی لاگ قرار دارد و قسمت هایی از لاگ که متغیر است به صورت * می‌آید.

Id	Content	EventId	EventTemplate
774	Notification time out: 3200	E31	Notification time out: <*>
493	Received connection request /10.10.34.11:45307	E40	Received connection request /<*>:<*>
688	Send worker leaving thread	E42	Send worker leaving thread
679	Interrupted while waiting for message on queue	E24	Interrupted while waiting for message on queue
688	Send worker leaving thread	E42	Send worker leaving thread
762	Connection broken for id 188978561024, my id = 1, error =	E11	Connection broken for id <*>, my id = <*>, error =
493	Received connection request /10.10.34.13:57707	E40	Received connection request /<*>:<*>

شکل ۳۰ توضیح برخی از ستون‌های دیتاست zookeeper

از آن جهت که اطلاعات متغیر در لاگ برای کاربر مهم خواهد بود، برای آموزش و مصورسازی مدل از ستون content استفاده خواهد شد.

به طور مثال پس از یافتن داده ناهنجار کاربر باید بتواند ویژگی‌های آن داده مانند ip ، port و ... را نیز ببیند. مانند شکل ۳۱ که یک نمونه ip به همراه port موردنظر آن ، مشاهده می‌شود و از آنجا که این کلمات در قسمت مربوط به رنگ قرمز می‌باشد، نشانگر آن است که به احتمال زیاد کلمات در یک نمونه داده با برچسب warn یا هشدار قرار دارد، البته ممکن است یک لاگ شامل کلماتی با برچسب‌های متنوع باشد که در این گونه موارد پیش‌بینی کلی لاگ نیز گاهی اشتباه خواهد بود.



شکل ۳۱ نمایش اطلاعات لاگ با جزئیات مقادیر

برچسب‌های این دیتاست شامل دو گروه هشدار و اطلاعات می‌باشد که در ستون level در این دیتاست قرار دارد.

۳-۸-۲- آموزش و پیش‌بینی مدل

برای آموزش این دیتاست از مدل lstm که در بخش‌های قبل توضیح دادیم، استفاده می‌کنیم با این تفاوت که سائز بیشینه توکن برای هر نمونه از داده‌ها را ۱۰ در نظر گرفته‌ایم که این مورد باتوجه به کوتاه بودن متن لاگ‌ها نسبت به دیتاست قبل که نظرات پیرامون فیلم بود، است.

برای مدیریت نمونه‌هایی که سائز آنها کمتر یا بیشتر از ۱۰ است از تابع pad_sequences کتابخانه keras استفاده شده است.

از آن جهت که داده‌های آموزش و تست پیش‌تر جدا نشده‌اند با توابع موجود در کتابخانه sklearn داده‌ها را به دو دسته تست و آموزش تقسیم کرده‌ایم.

ساختار مدل در شکل ۳۲ و دقت و خطای مدل در هر epoch در شکل ۳۳ آورده شده است.


```
summary of model_lstm
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 10, 32)	30432
lstm_1 (LSTM)	(None, 10, 100)	53200

```
Total params: 83632 (326.69 KB)
```

```
Trainable params: 83632 (326.69 KB)
```

```
Non-trainable params: 0 (0.00 Byte)
```

```
None
```

```
1/4 [====>.....] - ETA: 0s
```

```
4/4 [=====] - 0s 1ms/step
```

```
1/4 [====>.....] - ETA: 0s
```

```
4/4 [=====] - 0s 288us/step
```

```
hidden state shape :
```

```
(1000, 100)
```

```
summary of model_pred
```

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
dropout_1 (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 1)	101

```
Total params: 101 (404.00 Byte)
```

```
Trainable params: 101 (404.00 Byte)
```

```
Non-trainable params: 0 (0.00 Byte)
```

شکل ۳۲ ساختار مدل lstm به تفکیک دو زیر مدل مجزای آن شامل مصورسازی و پیش‌بینی


```

1/23 [>.....] - ETA: 15s - loss: 0.6921 - accuracy: 0.5469
12/23 [=====>.....] - ETA: 0s - loss: 0.5091 - accuracy: 0.7096
22/23 [=====>.....] - ETA: 0s - loss: 0.3054 - accuracy: 0.8352
23/23 [=====] - 1s 14ms/step - loss: 0.2991 - accuracy: 0.8389 - val_loss: -0.0420 - val_accuracy: 0.9875
Epoch 2/5

1/23 [>.....] - ETA: 0s - loss: 0.0284 - accuracy: 1.0000
11/23 [=====>.....] - ETA: 0s - loss: 0.0113 - accuracy: 0.9972
21/23 [=====>.....] - ETA: 0s - loss: -0.0322 - accuracy: 0.9926
23/23 [=====] - 0s 6ms/step - loss: -0.0372 - accuracy: 0.9924 - val_loss: -0.1221 - val_accuracy: 0.9875
Epoch 3/5

1/23 [>.....] - ETA: 0s - loss: 0.0079 - accuracy: 1.0000
11/23 [=====>.....] - ETA: 0s - loss: -0.0623 - accuracy: 0.9929
21/23 [=====>.....] - ETA: 0s - loss: -0.0469 - accuracy: 0.9948
23/23 [=====] - 0s 6ms/step - loss: -0.0581 - accuracy: 0.9937 - val_loss: -0.1612 - val_accuracy: 0.9875
Epoch 4/5

1/23 [>.....] - ETA: 0s - loss: 0.0030 - accuracy: 1.0000
11/23 [=====>.....] - ETA: 0s - loss: -0.0740 - accuracy: 0.9943
21/23 [=====>.....] - ETA: 0s - loss: -0.0781 - accuracy: 0.9940
23/23 [=====] - 0s 6ms/step - loss: -0.0728 - accuracy: 0.9944 - val_loss: -0.1813 - val_accuracy: 0.9875
Epoch 5/5

1/23 [>.....] - ETA: 0s - loss: 0.0031 - accuracy: 1.0000
11/23 [=====>.....] - ETA: 0s - loss: -0.0794 - accuracy: 0.9943
21/23 [=====>.....] - ETA: 0s - loss: -0.0901 - accuracy: 0.9940
23/23 [=====] - 0s 6ms/step - loss: -0.0840 - accuracy: 0.9944 - val_loss: -0.1993 - val_accuracy: 0.9875
13/13 - 0s - loss: -1.1928e-01 - accuracy: 0.9925 - 25ms/epoch - 2ms/step
Accuracy: 99.25%

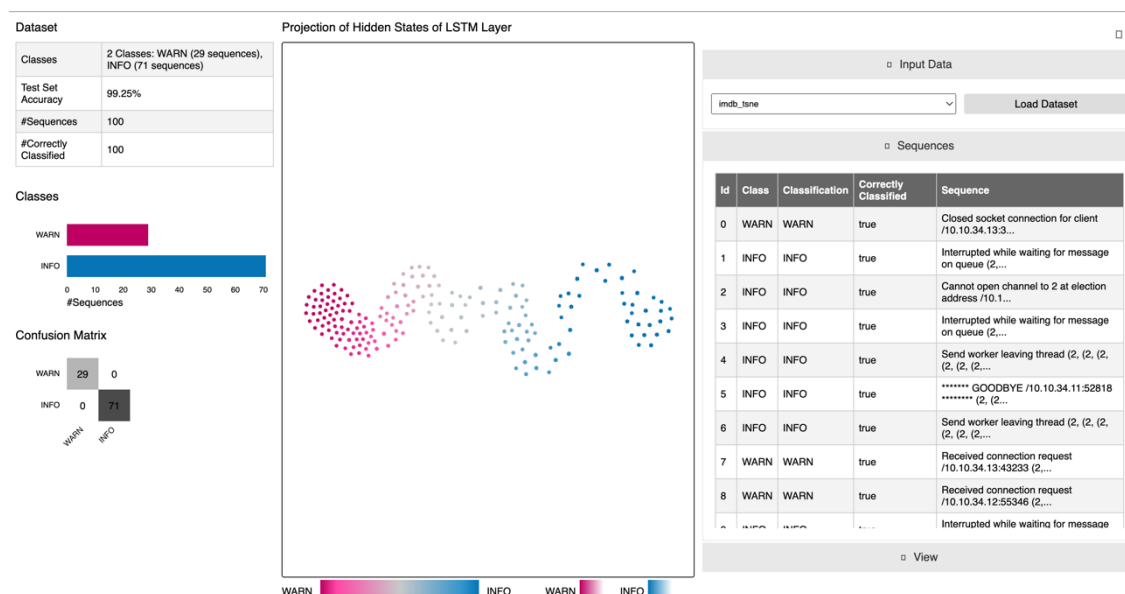
```

شکل ۳۳ آموزش مدل lstm بر روی دیتاست لاگ

۳-۸-۳- مصورسازی مدل

مصورسازی نهایی کلمات موجود در ۱۰۰ نمونه لاگ در این دیتاست از بخش تست در شکل ۳۴ آورده شده است.

مصورسازی نهایی به گونه ای است که کلمات با برچسب های همگون به خوبی در فضا قرار گرفته اند و کاربر می تواند با قراردادن موس خود در قسمت قرمز رنگ، کلماتی که برچسب هشدار دارند و به احتمال زیاد در لاگ با برچسب هشدار نیز قرار گرفته اند، را ملاحظه کند و اقدامات امنیتی متناسب را اتخاذ کند.



شکل ۳۴ مصورسازی ۱۰۰ داده نمونه تست از دیتاست zookeeper

۳-۸-۴- نتایج به دست آمده از مدل

دو نمونه از مصورسازی لاگ را در این نمونه از مصورسازی برای لاگ‌هایی با برجسب هشدار و اطلاعات در شکل‌های ۳۵ و ۳۶ آورده شده‌اند.

در این مصورسازی کلمات موجود در لاگ با خط به یکدیگر وصل شده‌اند و شدت رنگ هر کلمه یا خط میان کلمات نشان‌دهنده درجه اطمینان مدل از برجسب پیش‌بینی شده می‌باشد. به طور مثال نواحی خاکستری رنگ شامل کلمات کم‌اهمیت در پیش‌بینی مدل خواهند بود.

در شکل ۳۵ لاگ موردنظر برجسب هشدار دارد و ip, port که در این لاگ آورده شده‌اند در وسط تصویر دیده می‌شود و کاربر می‌تواند در صورت لزوم از آنها استفاده کند. توجه کنید که مقادیر ip, port به رنگ خاکستری نیستند و به علت قرارگیری موس روی آنها موقتاً خاکستری شده‌اند. در شکل ۳۷ رنگ اصلی این مقادیر قابل مشاهده است.

Sequence Overview

Sequence	24
Class	INFO
Classification	INFO
Correctly Classified	true
Confidence	100.0%

Current Sequence

Cannot open channel to 3 at election
address /10.10.34.13:3888

Class Contribution

Word id

Hidden State Distances

Word id

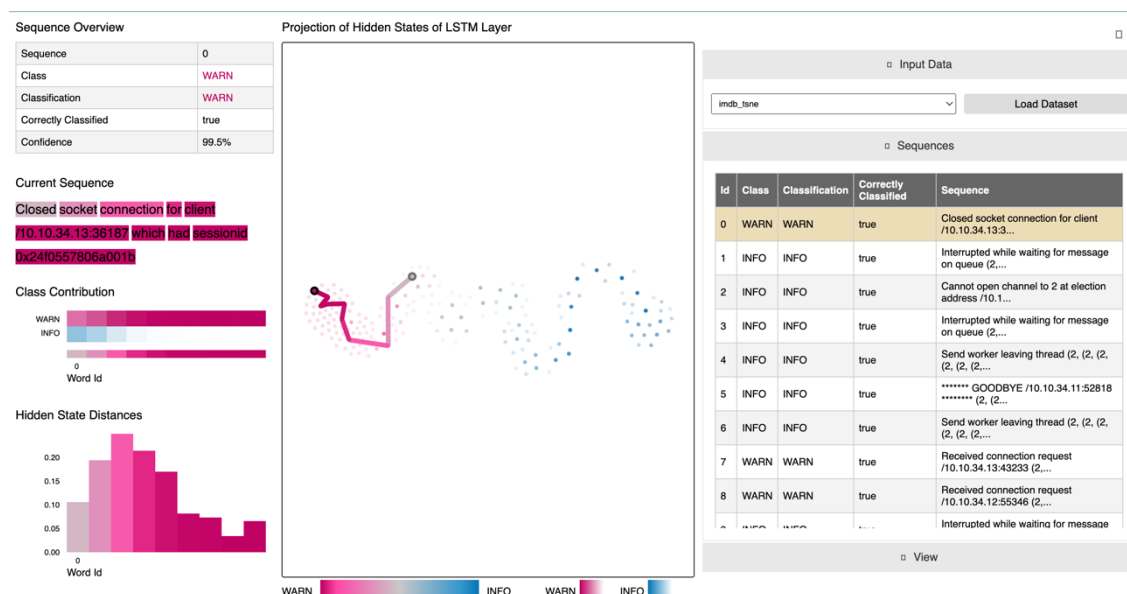
Projection of Hidden States of LSTM Layer

Input Data

Load Dataset

Sequences

Index	WARN	INFO	True	Message
22	WARN	WARN	true	/10.10.34.13:57895 (2,...
23	INFO	INFO	true	Interrupted while waiting for message on queue (2,...
24	INFO	INFO	true	Cannot open channel to 3 at election address /10.1... (2,...
25	INFO	INFO	true	Interrupting SendWorker (2, (2, (2, (2, (2, (2,...
26	INFO	INFO	true	Interrupting SendWorker (2, (2, (2, (2, (2, (2,...
27	INFO	INFO	true	Interrupted while waiting for message on queue (2,...
28	INFO	INFO	true	Send worker leaving thread (2, (2, (2, (2, (2, (2,...
29	WARN	WARN	true	Expiring session 0x14ed93111200d2, timeout of 100...
30	INFO	INFO	true	Send worker leaving thread (2, (2, (2, (2, (2, (2,...
31	INFO	INFO	true	broken for id 188978561024, my id = 3, error =...
32	WARN	WARN	true	Received connection request /10.10.34.12:54830 (2,...



شکل ۳۷ مصورسازی کلمات درون لاگ به همراه شدت رنگ آنها

۳-۹- جمع بندی

در این فصل ابتدا ماتریس شباهت و الگوریتم svm را پیاده سازی کردیم. سپس به سراغ شبکه های عصبی و مصورسازی ابرپارامترهای آن رفتیم. با مقایسه این دو مصورسازی به این نتیجه رسیدیم که مصورسازی ابرپارامترها و معادل سازی آن با کلمات درون لاگ، برای کاربر بسیار کاربردی تر از حالت دیگر است.

فصل چهارم

نتایج

۴-۱- مقدمه

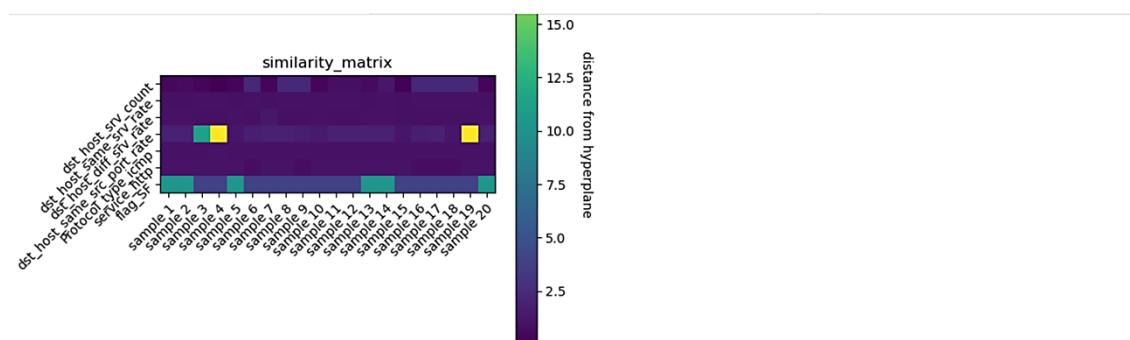
در این فصل نتایج به دست آمده در مدل‌ها و مصورسازی‌های صورت گرفته را به تفکیک مورد بررسی قرار خواهیم داد.

ابتدا بررسی مصورسازی داده‌های تست با ماتریس شباهت و مدل svm را انجام می‌دهیم، سپس مدل‌های مبتنی بر شبکه عصبی و مصورسازی به کمک t-sne را پیش رو داریم و در نهایت در پایان فصل نتایج مدل نهایی را شرح می‌دهیم.

۴-۲- نتایج مصورسازی با ماتریس شباهت

در شکل ۳۸ مقادیری که این ماتریس برای داده‌های ۳ و ۴ در ویژگی dst-host-same-src-port-rate محاسبه کرده نسبت به بقیه داده‌ها بیشتر بوده و همان‌طور که مقدار این ویژگی را برای این دو داده در پایین تصویر نیز مشاهده می‌کنید، این دو داده نسبت به سایر داده‌ها در این ویژگی به صورت ناهنجار عمل می‌کنند. البته ممکن است این مقادیر ناهنجار نباشند و جزو داده‌هایی باشند که مدل هنوز آنها را ندیده است و در داده‌های آموزشی مدل قرار نداشته، در هر صورت این دو داده می‌توانند به عنوان کاندید برای داده‌های ناهنجار به همراه جزئیات موجود در لاگ آنها به متخصص شبکه داده شود تا بررسی‌های بیشتر روی آن صورت گیرد. مقادیر نمونه‌ها در برخی از ویژگی‌ها نیز بسیار متنوع است؛ مانند ویژگی flag-sf در شکل ۳۸. در مورد این ویژگی نمی‌توان به طور کلی و جامع نظر داد چرا که نمونه‌ها در این ویژگی مقادیر بسیار متفاوتی نسبت به یکدیگر دارند. هم چنین روند یا الگوی مشخصی را نیز نمی‌توان برای مقادیر ماتریس پیدا کرد. یکی از نتایج مناسبی که این ماتریس ارائه کرده است در داده شماره ۷ با ویژگی dst-host-diff-srv-rate

می‌باشد، مقدار این ویژگی برای این نمونه لاگ دو و خورده‌ای است و نسبت به سایر نمونه‌ها این مقدار زیاد است؛ اما در ماتریس شباهت این مقدار با رنگ متفاوتی نشان داده نمی‌شود که به این علت است که فاصله منتهی این نمونه با مرز تصمیم‌گیر SVM کم است، مرز تصمیم‌گیر تمام داده‌ها را آموزش دیده است؛ اما در اینجا تنها مقادیر این ویژگی برای ۱۰ نمونه از داده‌ها آورده شده است و نمی‌توان تصمیم گرفت که این مقدار برای نمونه شماره ۷ ناهنجار محسوب می‌شود یا خیر.



<Figure size 5000x1000 with 0 Axes>

```
log_dataset.head(10)
```

	src_bytes	dst_bytes	hot	count	srv_count	same_srv_rate	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_same_src_port_rate
0	-0.021988	-0.096896	-0.113521	1.166764	-0.237191	-1.697859	-1.169697	-1.305370	-0.138370	-0.431856
1	-0.021988	-0.096896	-0.113521	0.443234	-0.338246	-1.770589	-1.250212	-1.397181	-0.138370	-0.431856
2	0.005473	-0.096896	-0.113521	-0.607053	-0.338246	0.629488	-0.489800	0.002934	-0.228985	1.559906
3	-0.021946	-0.096896	-0.113521	-0.607053	0.380366	0.629488	-0.749234	0.898090	-0.410217	2.833328
4	-0.021988	-0.096189	-0.113521	-0.607053	-0.259648	0.629488	-0.489800	-0.685647	0.360018	-0.333901
5	-0.021423	0.587166	-0.113521	-0.583713	-0.304561	0.629488	1.022079	0.898090	-0.410217	-0.399204
6	-0.019826	-0.078657	-0.113521	-0.607053	-0.315789	0.629488	-1.008669	-1.144701	2.851953	-0.431856
7	-0.021715	-0.088696	-0.113521	-0.607053	-0.338246	0.629488	1.022079	0.898090	-0.410217	-0.431856
8	-0.021296	-0.074887	-0.113521	-0.358096	0.178256	0.629488	1.022079	0.898090	-0.410217	-0.399204
9	-0.021933	-0.089497	-0.113521	-0.607053	-0.338246	0.629488	-1.026561	-0.249546	-0.047754	-0.366553

شکل ۳۸ بررسی نتایج ماتریس شباهت

۳-۴- نتایج مصورسازی حالت‌های پنهان

همان‌طور که در فصل پیشین گفته شد در صورت مصورسازی کلمات موجود در لاگ‌ها به صورت خام و تنها به کمک الگوریتم‌های کاهش بعد مانند t-sne نمی‌توان نتیجه مطلوبی دریافت کرد. در واقع بهتر است برای مصورسازی در این حالت از شبکه‌های عصبی برای آموزش داده‌های لاگ استفاده کرد و سپس ابرپارامترهای شبکه را با کلمات موجود در لاگ‌ها معادل‌سازی کرد، در پایان نیز به کمک t-sne کلمات را در یک فضای دوبعدی ترسیم کرد.

شبکه‌های عصبی که در فصل قبل پیاده‌سازی کردیم شامل لایه‌های lstm, rnn بود؛ اما با قراردادن لایه‌های دیگر در شبکه نیز نتایج مشابه دریافت کردیم. به‌طور کلی می‌توان نتیجه گرفت که اکثر لایه‌های بازگشتی

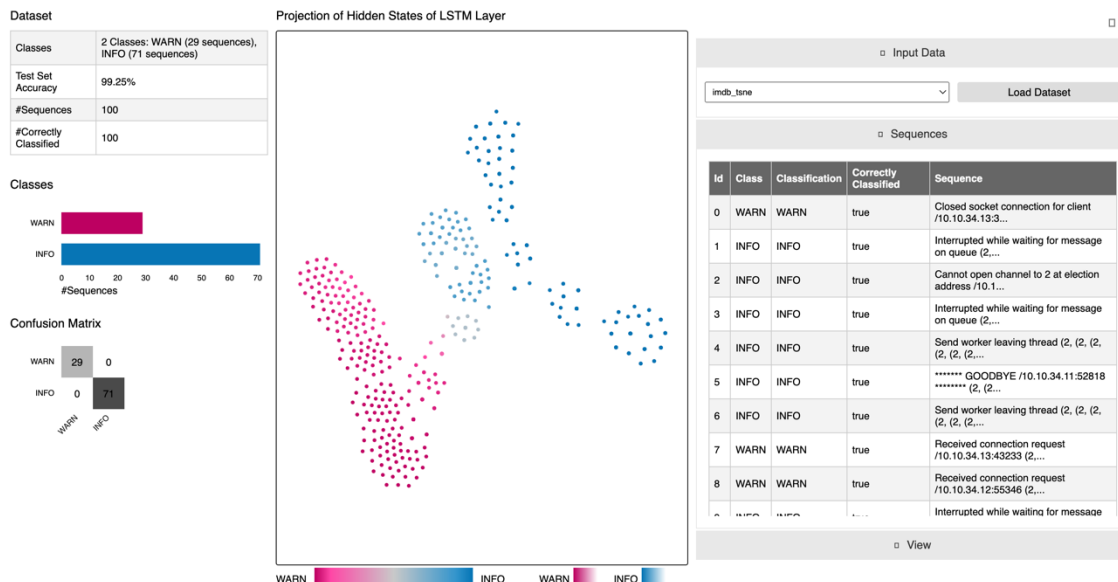
موجود در کتابخانه keras گزینه‌های مناسبی برای جایگزینی لایه‌ای که در پی مصورسازی حالت‌های پنهان آن هستید، خواهد بود. چند نمونه از این لایه‌ها به همراه مصورسازی آنها در ادامه آورده شده است.

مصورسازی حالت‌های پنهان در لایه دوجهته^۱. (شکل ۳۹)

مصورسازی حالت‌های پنهان در لایه gru^2 . (شکل ۴۰)

مصورسازی حالت‌های پنهان در لایه پیچشی^۳. (شکل ۴۱) در این حالت مصورسازی از لحاظ دسته‌بندی کلمات مرتبط به هم با برجسب یکسان بسیار مفیدتر از حالات دیگر دیده شد. ساختار مدل در این شبکه عصبی پیچشی در شکل ۴۲ آورده شده است.

برای مقایسه نتایج به‌دست‌آمده از t-sne در شکل ۴۳ مصورسازی به کمک الگوریتم کاهش بعد دیگری به نام pca انجام گرفته است که بیان می‌دارد به جز تفاوت ظاهری، تفاوت چشمگیر دیگری نمی‌توان میان این دو الگوریتم قائل شد.

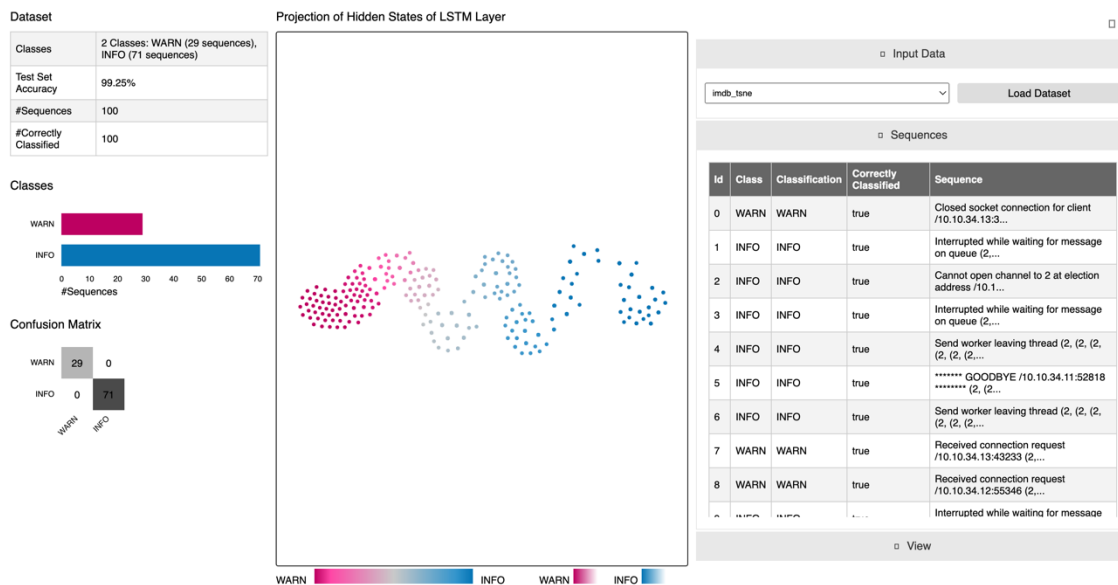


شکل ۳۹ مصورسازی حالت‌های پنهان لایه Bidirectional

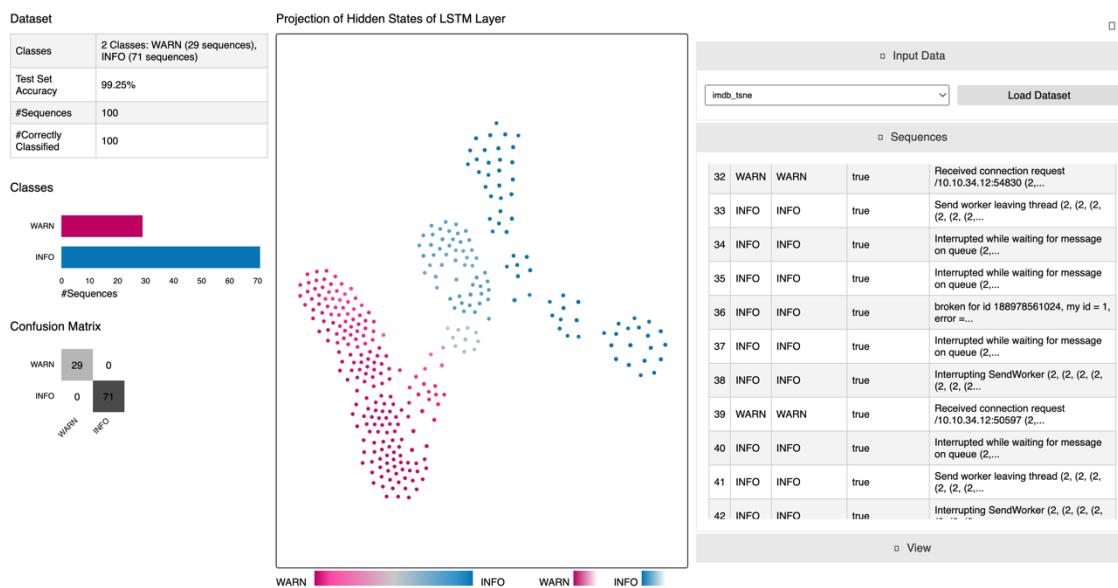
^۱ Bidirectional

^۲ Gated recurrent units

^۳ convolutional



شکل ۴۰ مصورسازی حالت‌های پنهان در لایه gru



شکل ۴۱ مصورسازی حالت‌های پنهان لایه پیچشی


```

Accuracy: 98.50%
summary of model_lstm
Model: "sequential_1"

```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 10, 32)	30432
conv1d_1 (Conv1D)	(None, 8, 32)	3104
flatten_1 (Flatten)	(None, 256)	0

```

=====
Total params: 33536 (131.00 KB)
Trainable params: 33536 (131.00 KB)
Non-trainable params: 0 (0.00 Byte)
None
1/4 [=====>.....] - ETA: 0s
4/4 [=====] - 0s 325us/step

1/4 [=====>.....] - ETA: 0s
4/4 [=====] - 0s 269us/step
hidden state shape :
(100, 256)
summary of model_pred
Model: "sequential_3"

```

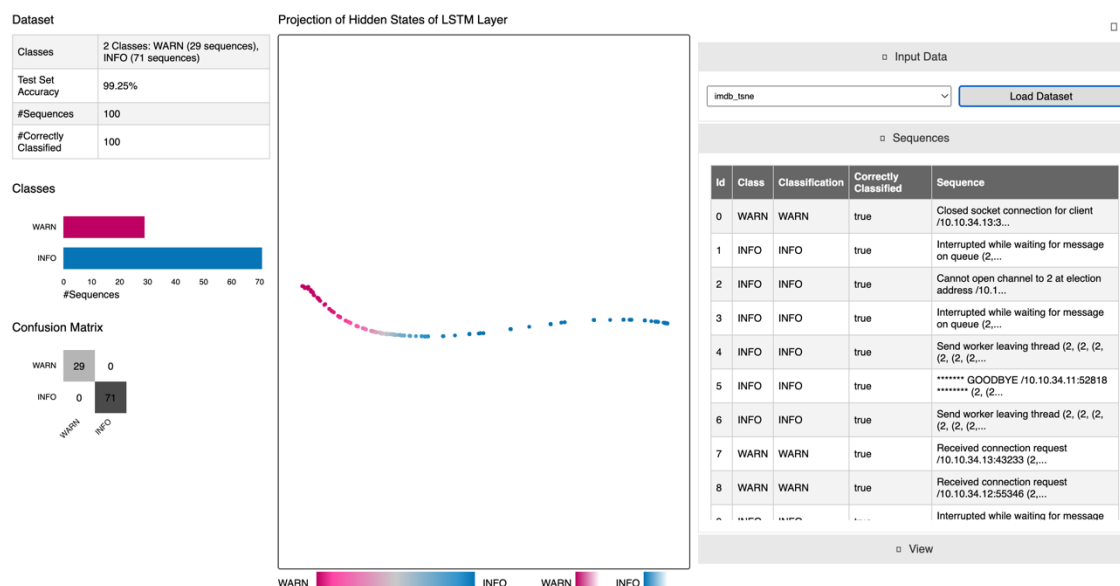
Layer (type)	Output Shape	Param #
dropout_1 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 1)	257

```

=====
Total params: 257 (1.00 KB)
Trainable params: 257 (1.00 KB)
Non-trainable params: 0 (0.00 Byte)

```

شکل ۴۲ ساختار مدل شبکه عصبی پیچشی به همراه آموزش و دقت آن



شکل ۴۳ مصورسازی حالت‌های پنهان با الگوریتم کاهش بعد pca

۴-۴- جمع‌بندی

با بررسی نتایج به‌دست‌آمده در این دو روش مصورسازی، دیدیم که هر دو روش تا حدی کاربردی هستند و نیاز کاربر را رفع خواهند کرد، اما به‌طور کلی نوع شبکه و عملکردی که دارد، نوع لاگی که در شبکه تولید می‌شود و متخصصین شبکه که قرار است از این مصورسازی‌ها استفاده کنند از عواملی هستند که در انتخاب روش مناسب برای مصورسازی مؤثر هستند.

اگر نمونه لاگی که نودهای شبکه تولید می‌کنند ساختار متنی و ترتیبی دارد؛ اما داده‌ی بدون ساختار است، مصورسازی کلمات درون لاگ‌ها به کمک حالت‌های پنهان شبکه‌های عصبی بازگشتی، می‌تواند مفید واقع شود.

اما در صورتی که بتوان ویژگی‌های تکراری و مشخصی را از لاگ‌ها استخراج کرد و آنها را به‌صورت داده‌های ساختاریافته^۱ ذخیره کرد، می‌توان از ماتریس شباهت برای مشاهده میزان شباهت ویژگی‌ها با مدل svm استفاده کرد.

^۱ structural data

فصل پنجم

نتیجه‌گیری و پیشنهادها

۵-۱- پیشنهادات در صورت استفاده از ماتریس شباهت

پس از بیان نتایج ماتریس شباهت، ممکن است کاربران شبکه بخواهند از مزایای این گونه مصورسازی بهره‌مند شوند؛ اما چالش بسیار مهمی که پیش روی متخصصین شبکه در ترسیم آن خواهد بود، این است که نمی‌توان بازه‌های عددی متفاوتی برای مقادیر ماتریس داشت. به طور مثال ممکن است مقادیر ماتریس که همان فاصله منتهی ویژگی با مرز تصمیم‌گیر است، ۲۰ به بالا باشد و این مقدار برای یک ویژگی می‌تواند ناهنجار نباشد؛ اما مقادیر ماتریس برای ویژگی دیگر کمتر از ۵ باشد و این فاصله از مرز تصمیم‌گیر منجر به بروز ناهنجاری شود.

در صورت ترسیم ماتریس شباهت این دو ویژگی کنار هم نمی‌توان تفاوت رنگ را در ماتریس مشاهده کرد و باید به طور جداگانه ماتریس این ویژگی‌ها رسم شوند.

برای یافتن این گونه ویژگی‌ها بهتر است متخصصین شبکه تحلیل‌های مناسبی را روی داده‌های آموزش داشته باشند و ماتریس شباهت را ابتدا برای دیتاست آموزش رسم کنند تا بتوانند دیتاست تست را بهتر به کمک این ماتریس تحلیل کنند.

۵-۲- پیشنهادات در صورت استفاده از مصورسازی حالت‌های پنهان

باتوجه به نتایج به‌دست‌آمده از مصورسازی حالت‌های پنهان در لایه‌های مختلف یک شبکه عصبی، در صورتی که کاربر بخواهد از این مدل مصورسازی استفاده کند، می‌تواند از مدل‌های مختلف و شبکه‌هایی با

ساختار گوناگون استفاده کند و یا حتی شبکه خود را با افزایش لایه‌های آن عمیق‌تر کند تا نتایج بهتری کسب کند. تنها باید در نظر داشته باشد که از لایه‌هایی با ساختار بازگشتی یا پیچشی استفاده کند و ابرپارامترهای این گونه لایه‌ها را مصورسازی کند، تا بتواند نتایج مناسبی از نقاط مصور شده، کسب کند. در غیر این صورت نمی‌تواند کلمات موجود در لاگ‌ها را به دسته‌بندی مناسب برای متخصصین شبکه مصورسازی کند.

منابع:

[١] A. Frei and M. Rennhard, "Histogram Matrix: Log File Visualization for Anomaly Detection," *2008 Third International Conference on Availability, Reliability and Security*, Barcelona, Spain, 2008, pp. 610-617, doi: 10.1109/ARES.2008.148.

[٢] S. Venkatraman and M. Alazab, "Use of Data Visualization for Zero-Day Malware Detection," *Security and Communication Networks*, vol. 2018, Article ID 1728303, pp. 1-13, 2018. doi: 10.1155/2018/1728303.

[٣] Y. Ming, et al., "Understanding Hidden Memories of Recurrent Neural Networks," in *2017 IEEE Conference on Visual Analytics Science and Technology (VAST)*, Phoenix, AZ, USA, 2017 pp. 13-24. doi: 10.1109/VAST.2017.8585721

[٤] R. Garcia, T. Munz, and D. Weiskopf, "Visual analytics tool for the interpretation of hidden states in recurrent neural networks," *Vis. Comput. Ind. Biomed. Art*, vol. 4, p. 24, 2021. doi: 10.1186/s42492-021-00090-0.

[٥] <https://www.unb.ca/cic/datasets/nsl.html>

[٦] <https://github.com/logpai/loghub>